

[NAME DER UNIVERSITÄT]

[NAME DER FAKULTÄT]

[NAME DES LEHRSTUHLs/INSTITUTS]

Bachelorarbeit

Erweiterung des Tensor Power Flow um PV-Knoten: Methoden, Implementierung und Vergleich mit konventionellen Lastflussverfahren

Verfasser: [Vorname Nachname]

Matrikelnummer: [Matrikelnummer]

Studiengang: [Studiengang]

Erstprüfer: [Prof. Dr.-Ing. Name]

Zweitprüfer: [Prof. Dr.-Ing. Name]

Betreuer: [M.Sc. Name]

Beginn: [Datum]

Abgabe: [Datum]

[Stadt], [Monat] [Jahr]

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Bachelorarbeit selbstständig und ohne unzulässige Hilfe Dritter verfasst habe. Alle verwendeten Quellen und Hilfsmittel sind angegeben.

Ort, Datum

Unterschrift

Kurzfassung

Abstract

Inhaltsverzeichnis

Eidesstattliche Erklärung	i
Kurzfassung	iii
Abstract	v
Abbildungsverzeichnis	xi
Tabellenverzeichnis	xiii
Symbolverzeichnis	xv
Abkürzungsverzeichnis	xvii
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung und Forschungsfragen	1
1.4 Aufbau der Arbeit	1
2 Grundlagen der Lastflussberechnung	3
2.1 Netzmodell und Knotentypen	3
2.1.1 Die Admittanzmatrix	3
2.1.2 Komplexe Leistung und Spannungsdarstellung	4
2.1.3 Knotentypen	4
2.1.4 Das ZIP-Lastmodell	5
2.2 Newton-Raphson-Verfahren	6
2.2.1 Die polare Standard-Formulierung	6
2.2.2 Unbekannte und Gleichungen	6
2.2.3 Iterationsvorschrift	7
2.2.4 Die Jacobi-Matrix in Blockform	7
2.2.5 Elemente der Jacobi-Matrix	8
2.2.6 Das vollständige Newton-Raphson-System	9

2.2.7	Behandlung von PV-Knoten	9
2.2.8	Q-Limits und PV-zu-PQ-Umwandlung	10
2.2.9	Konvergenzeigenschaften	10
2.2.10	Grenzen des Newton-Raphson-Verfahrens	11
2.3	Current Injection Method und PV-Behandlung	11
2.3.1	Strommismatch-Formulierung	12
2.3.2	Jacobi-Matrix der CIM	13
2.3.3	Behandlung von PV-Knoten in der CIM	15
2.3.4	Q-Limits und PV-zu-PQ-Umwandlung	17
2.4	Fixed-Point-Iteration und Konvergenz	17
2.4.1	Laurent-Reihen-Approximation	18
2.4.2	Herleitung des FPI-Algorithmus	18
2.4.3	Die Successive Approximation Method (SAM)	19
2.4.4	Konvergenzbeweis mittels Banach-Fixpunktsatz	20
2.4.5	Physikalische Interpretation der Konvergenzbedingung	21
3	Der Tensor Power Flow	23
3.1	Grundalgorithmus	23
3.2	Dense- und Sparse-Formulierung	24
3.2.1	Motivation: Der Tensor der Lastdaten	25
3.2.2	Dense-Formulierung	25
3.2.3	Sparse-Formulierung	26
3.3	Limitation: Keine PV-Knoten	27
3.3.1	Warum PV-Knoten diese Annahme verletzen	28
4	Methoden zur PV-Knoten-Integration im Tensor Power Flow	29
4.1	Analyse der Auswirkung von PV-Knoten auf die FPI-Struktur	30
4.1.1	Welche Matrizen werden von Q-Änderungen beeinflusst?	30
4.1.2	Umschreibung der Iteration	31
4.1.3	Konsequenzen für den Konvergenzbeweis	31
4.2	Methode A: Äußere Q-Schleife	32
4.2.1	Grundidee	32
4.2.2	Herleitung der Q-Korrekturformel	32
4.2.3	Eigenschaften der Korrekturformel	34
4.2.4	Algorithmus	35
4.2.5	Rechentechnische Betrachtung	36
4.3	Methode B: Eingebettete Q-Korrektur	36
4.3.1	Grundidee	36
4.3.2	Mathematische Formulierung	37
4.3.3	Algorithmus	40

4.3.4	Eigenschaften und Diskussion	41
4.4	Tensorformulierung der Methoden	42
4.4.1	Recap: Tensor-Reshape und allgemeine FPI	42
4.4.2	Umformulierung für den Tensor-Fall	42
4.4.3	Tensorformulierung von Methode A	43
4.4.4	Tensorformulierung von Methode B	44
4.4.5	Gesamtvergleich der Tensor-Operationen	45
4.4.6	Zusammenfassung: Warum die Parallelisierung erhalten bleibt	45
5	Implementierung und Testdesign	47
5.1	Programmiersprache und Bibliotheken	47
5.2	Testnetze	47
5.3	Szenarien und Parameter	47
5.4	Referenzmethoden	47
5.5	Bewertungsmetriken	47
6	Ergebnisse und Diskussion	49
6.1	Validierung gegen Newton-Raphson-Referenz	49
6.2	Konvergenzverhalten	49
6.3	Rechenzeit: Skalierung mit Netzgröße	49
6.4	Rechenzeit: Skalierung mit Anzahl der Lastflüsse	49
6.5	Einfluss der Anzahl der PV-Knoten	49
6.6	GPU vs. CPU	49
6.7	Vergleich der drei Methoden	49
6.8	Vergleich mit konventionellen Verfahren	49
7	Zusammenfassung und Ausblick	51
7.1	Zusammenfassung der Ergebnisse	51
7.2	Bewertung der Methoden	51
7.3	Ausblick	51
8	Implementierung des Tensor Power Flow	53
8.1	Überblick: Vom Netzmodell zur Lösung	53
8.2	Phase 1: Vorberechnung der konstanten Größen	54
8.2.1	Berechnung der Bus-Impedanzmatrix Z_B	54
8.2.2	Berechnung des Konstantvektors	55
8.3	Phase 2: Initialisierung	55
8.3.1	Flat Start der Spannungen	56
8.3.2	Aufbau der Leerlaufspannungsmatri	56
8.3.3	Aufbau der konjugierten Leistungsmatrix $\dot{\mathbf{S}}^*$	57

8.3.4	Vorberechnung von	57
8.4	Phase 3: Die Iterationsschleife (Kern des TPF)	57
8.4.1	Schritt 3.1: Element-weise Inversion der konjugierten Spannung	58
8.4.2	Schritt 3.2: Berechnung des konjugierten Stroms (Hadamard- Produkt)	58
8.4.3	Schritt 3.3: Die Kern-Iteration (Matrix-Matrix-Multiplikation)	59
8.4.4	Zusammenfassung: Ein vollständiger Iterationsschritt	60
8.5	Phase 4: Konvergenzprüfung	61
8.5.1	Berechnung der aus den Spannungen resultierenden Leistung .	61
8.5.2	Berechnung des maximalen Mismatches	62
8.5.3	Abbruchbedingung	62
8.6	Die Vorzeichenkonvention im Detail	63
8.7	Die Tensor-Struktur: Warum Batch schneller ist	64
8.8	Zusammenfassung: Der vollständige Algorithmus	66
8.9	Code-zu-Mathematik-Referenztafel	66
8.10	Numerisches Beispiel: 2-Bus-System	67

Abbildungsverzeichnis

8.1	Die drei Teilschritte einer einzelnen FPI-Iteration im TPF.	60
-----	---	----

Tabellenverzeichnis

2.1	Klassifizierung der Knotentypen im Lastflussproblem.	5
2.2	Blöcke der Jacobi-Matrix in polarer Formulierung.	8
4.1	Abhängigkeit der FPI-Bestandteile von der Blindleistung Q_k eines PV-Knotens für den allgemeinen ZIP-Fall.	30
4.2	Kosten der Hilfsmatrizen-Aktualisierung in Methode B pro Iteration.	41
4.3	Rechentechnischer Vergleich der Tensor-Operationen pro Iteration (Spezialfall $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten).	45
8.1	Zuordnung zwischen Code-Variablen und mathematischen Symbolen.	67

Symbolverzeichnis

Abkürzungsverzeichnis

Kapitel 1

Einleitung

1.1 Motivation

1.2 Problemstellung

1.3 Zielsetzung und Forschungsfragen

1.4 Aufbau der Arbeit

Kapitel 2

Grundlagen der Lastflussberechnung

Die Lastflussberechnung (engl. *Power Flow*, PF) ist eine der fundamentalsten Analysen in der elektrischen Energietechnik. Sie dient der Bestimmung der stationären Spannungen und Ströme an allen Knoten eines Netzes unter gegebenen Erzeugungs- und Verbrauchsbedingungen. Dieses Kapitel führt die mathematischen Grundlagen ein, auf denen die in Kapitel 3 und 4 vorgestellten Methoden aufbauen.

2.1 Netzmodell und Knotentypen

2.1.1 Die Admittanzmatrix

Betrachtet wird ein Netz mit einem Einspeiseknoten (Umspannwerk), $b = |\Omega_d|$ Lastknoten und $\phi = |\Omega_\phi|$ Phasen. Die Beziehung zwischen Knotenspannungen und -strömen wird durch die Admittanzmatrix $\mathbf{Y} \in \mathbb{C}^{(b+1)\phi \times (b+1)\phi}$ beschrieben [1]:

$$\begin{bmatrix} \mathbf{i}_s \\ -\mathbf{i}_d \end{bmatrix} = \begin{bmatrix} \mathbf{Y}_{ss} & \mathbf{Y}_{sd} \\ \mathbf{Y}_{ds} & \mathbf{Y}_{dd} \end{bmatrix} \begin{bmatrix} \mathbf{v}_s \\ \mathbf{v}_d \end{bmatrix} \quad (2.1)$$

Dabei bezeichnen:

- $\mathbf{i}_s \in \mathbb{C}^{\phi \times 1}$: Vektor der komplexen Strominjektionen am Einspeiseknoten,
- $\mathbf{i}_d \in \mathbb{C}^{b\phi \times 1}$: Vektor der komplexen Strominjektionen an den Lastknoten $d \in \Omega_d$,
- $\mathbf{v}_s \in \mathbb{C}^{\phi \times 1}$: Vektor der komplexen Knotenspannungen am Einspeiseknoten,
- $\mathbf{v} := \mathbf{v}_d \in \mathbb{C}^{b\phi \times 1}$: Vektor der komplexen Knotenspannungen an den Lastknoten.

Die Admittanzmatrix ist in vier Blöcke unterteilt: $\mathbf{Y}_{ss}$ beschreibt die Selbstadmittanz des Einspeiseknotens, $\mathbf{Y}_{dd}$ die Verbindungen zwischen den Lastknoten untereinander,

und $\mathbf{Y}_{sd} = \mathbf{Y}_{ds}^T$ die Kopplung zwischen Einspeise- und Lastknoten. Die Formulierung in (2.1) ist allgemein und kann einphasige, mehrphasige, radiale sowie vermaschte Netze mit verteilter Erzeugung abbilden [1].

Die Admittanzmatrix wird aus den Leitungsimpedanzen des Netzes aufgebaut. Für eine Leitung zwischen Knoten i und j mit der Impedanz $z_{ij} = r_{ij} + jx_{ij}$ ergibt sich die Admittanz als $y_{ij} = 1/z_{ij}$. Die Matrixelemente sind:

$$Y_{ii} = \sum_{j \in \mathcal{N}_i} y_{ij} + y_{i,\text{shunt}} \quad (\text{Diagonalelemente}) \quad (2.2)$$

$$Y_{ij} = -y_{ij} \quad (\text{Außerdiagonalelemente}) \quad (2.3)$$

wobei $\mathcal{N}_i$ die Menge aller direkt mit Knoten i verbundenen Knoten bezeichnet und $y_{i,\text{shunt}}$ eine eventuelle Shunt-Admittanz am Knoten i ist.

2.1.2 Komplexe Leistung und Spannungsdarstellung

Die komplexe Scheinleistung an einem Knoten k ist definiert als:

$$s_k = P_k + jQ_k = v_k \cdot i_k^* \quad (2.4)$$

wobei P_k die Wirkleistung, Q_k die Blindleistung, v_k die komplexe Spannung und i_k^* der konjugiert komplexe Strom ist. Daraus folgt der Strom:

$$i_k = \left(\frac{s_k}{v_k} \right)^* = \frac{s_k^*}{v_k^*} = \frac{P_k - jQ_k}{v_k^*} \quad (2.5)$$

Die komplexe Spannung kann in zwei äquivalenten Formen dargestellt werden:

$$\text{Polarform: } V_k = |V_k| \cdot e^{j\theta_k} = |V_k| \angle \theta_k \quad (2.6)$$

$$\text{Kartesische Form: } V_k = e_k + jf_k \quad (2.7)$$

mit den Zusammenhängen $e_k = |V_k| \cos(\theta_k)$, $f_k = |V_k| \sin(\theta_k)$ und $|V_k| = \sqrt{e_k^2 + f_k^2}$. Beide Darstellungen finden in den nachfolgenden Verfahren Anwendung: Die Polarform wird klassisch im Newton-Raphson-Verfahren verwendet, die kartesische Form in der Current Injection Method und im Tensor Power Flow.

2.1.3 Knotentypen

Im Lastflussproblem werden die Netzknoten anhand der bekannten und gesuchten Größen in drei Typen eingeteilt. Tabelle 2.1 gibt eine Übersicht.

Tabelle 2.1: Klassifizierung der Knotentypen im Lastflussproblem.

Typ	Bekannt	Gesucht	Typische Anwendung
Slack ($V\delta$)	$ V , \theta$	P, Q	Umspannwerk, Referenzknoten
PQ	P, Q	$ V , \theta$	Verbraucher, PV-Wechselrichter mit $\cos \varphi$ -Regelung
PV	$P, V $	Q, θ	Synchrongeneratoren, Einspeiser mit Spannungsregelung

Slack-Knoten ($V\delta$ -Knoten). Der Slack-Knoten dient als Referenz für Spannung und Winkel im Netz. Sein Spannungsbetrag $|V_s|$ und sein Winkel θ_s sind vorgegeben. Die zugehörige Wirk- und Blindleistung ergibt sich als Resultat der Lastflussberechnung und gleicht die Leistungsbilanz des Netzes aus. Im Verteilnetz entspricht der Slack-Knoten typischerweise dem Umspannwerk.

PQ-Knoten. An PQ-Knoten sind die Wirkleistung P_k^{spec} und die Blindleistung Q_k^{spec} vorgegeben. Der Spannungsbetrag $|V_k|$ und der Winkel θ_k (bzw. e_k und f_k) sind die gesuchten Größen. Die meisten Knoten in einem Verteilnetz – Verbraucher, Wechselrichter mit festem Leistungsfaktor – werden als PQ-Knoten modelliert.

PV-Knoten. An PV-Knoten sind die Wirkleistung P_k^{spec} und der Spannungsbetrag $|V_k^{\text{spec}}|$ vorgegeben. Die Blindleistung Q_k ist unbekannt und muss so bestimmt werden, dass die Spannungsvorgabe eingehalten wird. PV-Knoten stellen eine besondere Herausforderung dar, da sie eine zusätzliche Nebenbedingung in das Gleichungssystem einführen:

$$|V_k|^2 = e_k^2 + f_k^2 = |V_k^{\text{spec}}|^2 \quad (2.8)$$

Aus (2.5) wird deutlich, warum PV-Knoten problematisch sind: Der Strom $i_k = (P_k - jQ_k)/v_k^*$ kann nicht direkt berechnet werden, da Q_k unbekannt ist. Im Gegensatz dazu ist bei PQ-Knoten die vollständige Leistung $s_k = P_k + jQ_k$ bekannt, sodass der Strom unmittelbar berechenbar ist.

2.1.4 Das ZIP-Lastmodell

In der Praxis hängt die aufgenommene Leistung einer Last vom Spannungsbetrag ab. Dies wird durch das ZIP-Modell berücksichtigt, das die Last als Kombination dreier Typen darstellt [2]:

$$\mathbf{S}_d = \left(\text{diag}(\alpha_P) + \text{diag}(\alpha_I |\mathbf{V}_d|) + \text{diag}(\alpha_Z |\mathbf{V}_d|^2) \right) \mathbf{S}_n \quad (2.9)$$

wobei S_n die nominale Leistung, und α_Z , α_I , α_P die Koeffizienten für die impedanz-, strom- und leistungskonstanten Anteile sind, mit $\alpha_Z + \alpha_I + \alpha_P = 1$. Der leistungskonstante Anteil (α_P) ist für die Nichtlinearität des Lastflussproblems verantwortlich.

2.2 Newton-Raphson-Verfahren

Das Newton-Raphson-Verfahren (NR) ist die am weitesten verbreitete Methode zur Lösung des Lastflussproblems in Übertragungs- und Verteilnetzen [3]. Es basiert auf einer Linearisierung der nichtlinearen Lastflussgleichungen mittels der Taylor-Reihe erster Ordnung und zeichnet sich durch quadratische Konvergenz aus.

2.2.1 Die polare Standard-Formulierung

In der klassischen polaren Darstellung werden die Leistungsgleichungen als Funktion der Spannungsbeträge $|V_k|$ und Spannungswinkel θ_k formuliert. Die komplexe Spannung an Knoten k ist dabei:

$$V_k = |V_k| \cdot e^{j\theta_k} \quad (2.10)$$

Aus der allgemeinen Leistungsbeziehung $S_k = V_k I_k^*$ und dem Zusammenhang $I_k = \sum_{m=1}^{n_b} Y_{km} V_m$ ergeben sich die Wirk- und Blindleistungsinjektionen an Knoten k zu [3]:

$$P_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (2.11)$$

$$Q_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (2.12)$$

wobei $G_{km} = \text{Re}(Y_{km})$ die Konduktanz und $B_{km} = \text{Im}(Y_{km})$ die Suszeptanz des Admittanzmatrix-Elements Y_{km} bezeichnen, und $\theta_k - \theta_m$ die Winkeldifferenz zwischen den Knoten k und m ist.

2.2.2 Unbekannte und Gleichungen

Die Struktur des Gleichungssystems hängt von den Knotentypen ab. Der Unbekanntenvektor $\Delta \mathbf{x}$ und der Mismatch-Vektor $\mathbf{g}(\mathbf{x})$ in der polaren Formulierung sind:

Unbekannte:

$$\Delta \mathbf{x} = \begin{bmatrix} \Delta \boldsymbol{\theta} \\ \Delta |\mathbf{V}| \end{bmatrix} \quad (2.13)$$

wobei $\Delta\boldsymbol{\theta} \in \mathbb{R}^{(n_b-1) \times 1}$ die Winkelkorrekturen an allen PQ- und PV-Knoten sind und $\Delta|\mathbf{V}| \in \mathbb{R}^{n_{PQ} \times 1}$ die Spannungsbetragskorrekturen **nur** an PQ-Knoten darstellen.

Mismatch-Gleichungen:

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} \Delta\mathbf{P} \\ \Delta\mathbf{Q} \end{bmatrix} = \mathbf{0} \quad (2.14)$$

wobei $\Delta\mathbf{P} \in \mathbb{R}^{(n_b-1) \times 1}$ die Wirkleistungsmismatches an allen PQ- und PV-Knoten und $\Delta\mathbf{Q} \in \mathbb{R}^{n_{PQ} \times 1}$ die Blindleistungsmismatches **nur** an PQ-Knoten sind. Die Mismatches sind definiert als:

$$\Delta P_k = P_k^{\text{spec}} - P_k(\boldsymbol{\theta}, |\mathbf{V}|) \quad \text{für } k \in \Omega_{PQ} \cup \Omega_{PV} \quad (2.15)$$

$$\Delta Q_k = Q_k^{\text{spec}} - Q_k(\boldsymbol{\theta}, |\mathbf{V}|) \quad \text{für } k \in \Omega_{PQ} \quad (2.16)$$

2.2.3 Iterationsvorschrift

Das NR-Verfahren linearisiert den Mismatch-Vektor $\mathbf{g}(\mathbf{x})$ um den aktuellen Arbeitspunkt $\mathbf{x}^{(k)}$ mittels einer Taylor-Entwicklung erster Ordnung:

$$\mathbf{g}(\mathbf{x}^{(k)} + \Delta\mathbf{x}) \approx \mathbf{g}(\mathbf{x}^{(k)}) + \mathbf{J}(\mathbf{x}^{(k)}) \cdot \Delta\mathbf{x} = \mathbf{0} \quad (2.17)$$

Daraus ergibt sich der Korrekturvektor:

$$\Delta\mathbf{x}^{(k)} = -\mathbf{J}(\mathbf{x}^{(k)})^{-1} \cdot \mathbf{g}(\mathbf{x}^{(k)}) \quad (2.18)$$

Die Zustandsgrößen werden iterativ aktualisiert gemäß:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)} \quad (2.19)$$

Der Algorithmus wird initialisiert mit dem *Flat Start*: $|V_k|^{(0)} = 1,0$ p.u. und $\theta_k^{(0)} = 0$ für alle Knoten (außer Slack). Die Iteration wird beendet, wenn $\max(|\Delta P_k|, |\Delta Q_k|) < \varepsilon$ mit typischen Toleranzen $\varepsilon = 10^{-6}$ bis 10^{-8} .

2.2.4 Die Jacobi-Matrix in Blockform

Die Jacobi-Matrix $\mathbf{J}$ enthält die partiellen Ableitungen der Mismatch-Funktionen nach den Unbekannten und wird in der polaren Formulierung in vier Blöcke unterteilt [3]:

$$\begin{bmatrix} \Delta\mathbf{P} \\ \Delta\mathbf{Q} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{H} & \mathbf{N} \\ \mathbf{M} & \mathbf{L} \end{bmatrix}}_{\mathbf{J}} \begin{bmatrix} \Delta\boldsymbol{\theta} \\ \Delta|\mathbf{V}| \end{bmatrix} \quad (2.20)$$

Die vier Untermatrizen haben folgende Bedeutung und Dimension:

Tabelle 2.2: Blöcke der Jacobi-Matrix in polarer Formulierung.

Block	Ableitung	Dimension	Physikalische Bedeutung
$\mathbf{H}$	$\frac{\partial \Delta P}{\partial \theta}$	$(n_b - 1) \times (n_b - 1)$	Einfluss der Winkel auf die Wirkleistung
$\mathbf{N}$	$\frac{\partial \Delta P}{\partial V }$	$(n_b - 1) \times n_{PQ}$	Einfluss der Beträge auf die Wirkleistung
$\mathbf{M}$	$\frac{\partial \Delta Q}{\partial \theta}$	$n_{PQ} \times (n_b - 1)$	Einfluss der Winkel auf die Blindleistung
$\mathbf{L}$	$\frac{\partial \Delta Q}{\partial V }$	$n_{PQ} \times n_{PQ}$	Einfluss der Beträge auf die Blindleistung

2.2.5 Elemente der Jacobi-Matrix

Die Elemente der vier Blöcke ergeben sich durch Ableitung der Leistungsgleichungen (2.11)–(2.12) nach θ_m und $|V_m|$. Es wird zwischen Diagonal- und Außerdiagonalelementen unterschieden.

Außerdiagonalelemente ($k \neq m$):

$$H_{km} = \frac{\partial P_k}{\partial \theta_m} = |V_k| |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (2.21)$$

$$N_{km} = \frac{\partial P_k}{\partial |V_m|} = |V_k| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (2.22)$$

$$M_{km} = \frac{\partial Q_k}{\partial \theta_m} = -|V_k| |V_m| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (2.23)$$

$$L_{km} = \frac{\partial Q_k}{\partial |V_m|} = |V_k| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (2.24)$$

Diagonalelemente ($k = m$):

$$H_{kk} = \frac{\partial P_k}{\partial \theta_k} = -Q_k - B_{kk} |V_k|^2 \quad N_{kk} = \frac{\partial P_k}{\partial |V_k|} = \frac{P_k}{|V_k|} + G_{kk} |V_k| \quad (2.25)$$

$$M_{kk} = \frac{\partial Q_k}{\partial \theta_k} = P_k - G_{kk} |V_k|^2 \quad L_{kk} = \frac{\partial Q_k}{\partial |V_k|} = \frac{Q_k}{|V_k|} - B_{kk} |V_k| \quad (2.26)$$

Herleitung der Diagonalelemente. Die kompakte Form der Diagonalelemente ergibt sich durch Einsetzen der Leistungsdefinitionen. Beispielhaft für H_{kk} :

$$H_{kk} = \frac{\partial P_k}{\partial \theta_k} = |V_k| \sum_{\substack{m=1 \\ m \neq k}}^{n_b} |V_m| [-G_{km} \sin(\theta_k - \theta_m) + B_{km} \cos(\theta_k - \theta_m)] \quad (2.27)$$

Aus (2.12) erkennt man, dass dieser Ausdruck mit $-Q_k - B_{kk}|V_k|^2$ identisch ist, da der Anteil $m = k$ in der Summe den Term $-B_{kk}|V_k|^2$ liefert. Somit folgt die kompakte Darstellung in (??).

Zusammenhang zwischen den Blöcken. Aus den Gleichungen (2.21)–(2.24) lässt sich erkennen, dass für die Außerdiagonalelemente gilt:

$$H_{km} = -M_{km}, \quad N_{km} = L_{km} \quad (k \neq m) \quad (2.28)$$

Diese Symmetrieeigenschaft wird in der *Fast Decoupled Power Flow*-Methode ausgenutzt, ist aber für die vorliegende Arbeit nicht weiter relevant.

2.2.6 Das vollständige Newton-Raphson-System

Das in jeder Iteration k zu lösende lineare Gleichungssystem lautet ausgeschrieben:

$$\begin{bmatrix} \mathbf{H}^{(k)} & \mathbf{N}^{(k)} \\ \mathbf{M}^{(k)} & \mathbf{L}^{(k)} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{\theta}^{(k)} \\ \Delta |\mathbf{V}|^{(k)} \end{bmatrix} = \begin{bmatrix} \Delta \mathbf{P}^{(k)} \\ \Delta \mathbf{Q}^{(k)} \end{bmatrix} \quad (2.29)$$

Dieses System wird typischerweise durch LU-Zerlegung der dünnbesetzten (sparse) Jacobi-Matrix gelöst. Die Dünnbesetztheit resultiert aus der Tatsache, dass H_{km} , N_{km} , M_{km} , L_{km} nur dann von null verschieden sind, wenn eine direkte Verbindung (Leitung) zwischen den Knoten k und m besteht, d. h. $Y_{km} \neq 0$.

2.2.7 Behandlung von PV-Knoten

In der polaren Formulierung werden PV-Knoten **implizit** durch die Dimensionsreduktion des Gleichungssystems behandelt:

1. Der Spannungsbetrag $|V_k|$ ist fest vorgegeben $\Rightarrow \Delta|V_k|$ ist *keine* Unbekannte $\Rightarrow$ die Spalte für $\Delta|V_k|$ entfällt in $\mathbf{N}$ und $\mathbf{L}$.
2. Die Blindleistung Q_k ist unbekannt $\Rightarrow \Delta Q_k$ kann nicht als Mismatch formuliert werden $\Rightarrow$ die Zeile für ΔQ_k entfällt in $\mathbf{M}$ und $\mathbf{L}$.

Das resultierende System hat weiterhin gleich viele Gleichungen wie Unbekannte:

$$\begin{array}{lcl} \text{Unbekannte:} & \underbrace{(n_b - 1)}_{\Delta \theta \text{ (PQ + PV)}} & + \underbrace{n_{PQ}}_{\Delta |V| \text{ (nur PQ)}} \\ \text{Gleichungen:} & \underbrace{(n_b - 1)}_{\Delta P \text{ (PQ + PV)}} & + \underbrace{n_{PQ}}_{\Delta Q \text{ (nur PQ)}} \end{array}$$

Die Blindleistung Q_k am PV-Knoten ergibt sich nach Konvergenz implizit aus (2.12):

$$Q_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (2.30)$$

2.2.8 Q-Limits und PV-zu-PQ-Umwandlung

Generatoren an PV-Knoten verfügen über begrenzte Blindleistungskapazität $Q_k \in [Q_k^{\min}, Q_k^{\max}]$. Nach jeder Iteration wird geprüft, ob die aus (2.30) berechnete Blindleistung innerhalb der Grenzen liegt:

$$Q_k^{\min} \leq Q_k \leq Q_k^{\max} \quad (2.31)$$

Falls ein Limit verletzt wird, erfolgt eine **PV-zu-PQ-Umwandlung**:

- Die Blindleistung wird auf das verletzte Limit fixiert: $Q_k = Q_k^{\min}$ oder $Q_k = Q_k^{\max}$.
- Der Knoten wird fortan als PQ-Knoten behandelt ($|V_k|$ wird frei, ΔQ_k wird als Gleichung hinzugefügt).
- Die Jacobi-Matrix erhält eine zusätzliche Zeile (ΔQ_k) und eine zusätzliche Spalte ($\Delta |V_k|$).

In nachfolgenden Iterationen wird geprüft, ob der Spannungsbetrag wieder innerhalb des vorgegebenen Werts liegt, und der Knoten gegebenenfalls zurück in den PV-Typ konvertiert wird.

2.2.9 Konvergenzeigenschaften

Das NR-Verfahren weist **quadratische Konvergenz** auf, d. h. der Fehler wird in jeder Iteration quadriert:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq c \cdot \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2 \quad (2.32)$$

wobei $\mathbf{x}^*$ die exakte Lösung und c eine Konstante ist. Dies führt typischerweise zu einer Konvergenz in 3–6 Iterationen, weitgehend unabhängig von der Netzgröße [3]. Ein wesentlicher Vorteil des NR-Verfahrens in polarer Form ist, dass die Kopplung zwischen allen Unbekannten ($\Delta\theta$, $\Delta|V|$) *simultan* im linearen Gleichungssystem (2.29) berücksichtigt wird. Dadurch wird die volle Wechselwirkung zwischen Wirk- und Blindleistung, Winkeln und Beträgen in jedem Schritt ausgenutzt.

2.2.10 Grenzen des Newton-Raphson-Verfahrens

Trotz seiner quadratischen Konvergenz weist das NR-Verfahren Nachteile auf, die bei der Berechnung großer Mengen von Lastflüssen relevant werden:

1. Die Jacobi-Matrix $\mathbf{J}(\mathbf{x}^{(k)})$ muss in *jeder* Iteration neu berechnet werden, da ihre Elemente (2.21)–(2.26) von den aktuellen Spannungswerten abhängen.
2. Die Konvergenz hängt vom Startwert ab; bei ungünstiger Initialisierung kann das Verfahren zur nicht-operationellen Lösung (niedrige Spannung, hoher Strom) konvergieren oder divergieren [1].
3. Eine Parallelisierung über mehrere Lastflüsse ist schwierig, da jeder Lastfluss seine eigene, iterationsabhängige Jacobi-Matrix besitzt.

Diese Einschränkungen motivieren die Suche nach alternativen Verfahren, insbesondere für Anwendungen, die Hunderttausende von Lastflüssen erfordern. Der in Kapitel 3 vorgestellte Tensor Power Flow adressiert genau diese Limitierungen, kann dafür aber keine PV-Knoten direkt abbilden.

2.3 Current Injection Method und PV-Behandlung

Die konventionelle Newton-Raphson-Lastflussberechnung basiert auf Leistungs-Mismatch-Gleichungen, die in Polarkoordinaten formuliert werden. Dabei müssen in jedem Iterationsschritt trigonometrische Funktionen (Sinus, Kosinus) ausgewertet und die Jacobi-Matrix vollständig neu berechnet werden, was insbesondere bei großen Netzen einen erheblichen Rechenaufwand darstellt. Die *Current Injection Method* (CIM), vorgestellt von da Costa et al. [4], reformuliert das Lastflussproblem auf Basis von Strommismatch-Gleichungen in kartesischen Koordinaten. Dieser Ansatz führt zu einer Jacobi-Matrix, deren Struktur identisch mit der $(2n \times 2n)$ -Knotenadmittanzmatrix ist und deren außerdiagonale Blöcke – mit Ausnahme der PV-Knoten – während des gesamten Iterationsprozesses konstant bleiben. Dadurch entfällt die Notwendigkeit, transzendente Funktionen auszuwerten, und es ergibt sich eine signifikante Reduktion der Rechenzeit von über 20 % gegenüber der konventionellen Formulierung [4].

Im Folgenden wird zunächst die Strommismatch-Formulierung für PQ-Knoten hergeleitet (Abschnitt 2.3.1), anschließend die Struktur der resultierenden Jacobi-Matrix erläutert (Abschnitt 2.3.2), die Erweiterung auf PV-Knoten beschrieben (Abschnitt 2.3.3) und schließlich die Behandlung von Blindleistungsgrenzen mit der PV-zu-PQ-Umwandlung dargestellt (Abschnitt 2.3.4).

2.3.1 Strommismatch-Formulierung

Ausgangspunkt der CIM ist die komplexe Strominjektion an einem Knoten k . Die Netzgleichung in Matrixform lautet:

$$\mathbf{I} = \mathbf{Y} \mathbf{V} \quad (2.33)$$

wobei $\mathbf{Y} \in \mathbb{C}^{n \times n}$ die Knotenadmittanzmatrix, $\mathbf{V} \in \mathbb{C}^n$ der Vektor der komplexen Knotenspannungen und $\mathbf{I} \in \mathbb{C}^n$ der Vektor der komplexen Strominjektionen ist.

Für einen PQ-Knoten k ist die geplante komplexe Leistungsinjektion $S_k^{\text{sp}} = P_k^{\text{sp}} + j Q_k^{\text{sp}}$ bekannt. Die geplante (scheduled) Strominjektion ergibt sich aus der Beziehung zwischen Leistung und Strom:

$$I_k^{\text{sp}} = \left(\frac{S_k^{\text{sp}}}{V_k} \right)^* = \frac{P_k^{\text{sp}} - j Q_k^{\text{sp}}}{V_k^*} \quad (2.34)$$

wobei $V_k^* = e_k - j f_k$ die konjugiert komplexe Spannung am Knoten k bezeichnet, mit dem Realteil e_k und dem Imaginärteil f_k .

Die berechnete Strominjektion am Knoten k ergibt sich aus der Knotenadmittanzmatrix:

$$I_k^{\text{calc}} = \sum_{m=1}^n Y_{km} V_m = \sum_{m=1}^n (G_{km} + j B_{km})(e_m + j f_m) \quad (2.35)$$

Der komplexe Strommismatch am Knoten k ist definiert als die Differenz zwischen geplanter und berechneter Strominjektion [4]:

$$\Delta I_k = I_k^{\text{sp}} - I_k^{\text{calc}} = \Delta \mathcal{A}_k + j \Delta \mathcal{M}_k \quad (2.36)$$

Durch Separation in Real- und Imaginärteil ergeben sich zwei reelle Gleichungen pro Knoten. Dabei lassen sich die Stromkomponenten über die Leistungs-Mismatches ΔP_k und ΔQ_k ausdrücken. Die Wirk- und Blindleistungs-Mismatches sind definiert als:

$$\Delta P_k = P_k^{\text{sp}} - P_k^{\text{calc}} \quad (2.37)$$

$$\Delta Q_k = Q_k^{\text{sp}} - Q_k^{\text{calc}} \quad (2.38)$$

wobei die berechneten Leistungen durch die Spannungen und den Stromvektor gegeben sind:

$$P_k^{\text{calc}} = e_k \mathcal{A}_k^{\text{calc}} + f_k \mathcal{M}_k^{\text{calc}} \quad (2.39)$$

$$Q_k^{\text{calc}} = f_k \mathcal{A}_k^{\text{calc}} - e_k \mathcal{M}_k^{\text{calc}} \quad (2.40)$$

Durch algebraische Umformung lassen sich die Strommismatch-Komponenten di-

rekt über die bekannten Leistungs-Mismatches und die aktuellen Spannungswerte berechnen [4]:

$$\Delta_{\mathcal{A}_k} = \frac{e_k \Delta P_k + f_k \Delta Q_k}{V_k^2} \quad (2.41)$$

$$\Delta_{\mathcal{M}_k} = \frac{f_k \Delta P_k - e_k \Delta Q_k}{V_k^2} \quad (2.42)$$

mit $V_k^2 = e_k^2 + f_k^2$. Diese Formulierung ist für PQ-Knoten direkt auswertbar, da sowohl ΔP_k als auch ΔQ_k bekannte Größen sind. Die Gleichungen (2.41) und (2.42) bilden das Gleichungssystem, welches im Newton-Raphson-Verfahren iterativ gelöst wird.

Die Lastmodellierung erfolgt gemäß dem in Abschnitt 2.1.4 eingeführten ZIP-Modell in polynomialer Form:

$$P_{L(k)} = P_{0k} (\alpha_Z V_k^2 + \alpha_I V_k + \alpha_P) \quad (2.43)$$

$$Q_{L(k)} = Q_{0k} (\alpha_Z V_k^2 + \alpha_I V_k + \alpha_P) \quad (2.44)$$

wobei die Koeffizienten die Bedingung $\alpha_Z + \alpha_I + \alpha_P = 1$ erfüllen müssen. Die Terme α_Z , α_I und α_P repräsentieren die Anteile der konstanten Impedanz-, konstanten Strom- und konstanten Leistungslast (vgl. Gleichung (2.9)).

2.3.2 Jacobi-Matrix der CIM

Das Newton-Raphson-Verfahren, angewandt auf die Strommismatch-Gleichungen, führt zu folgendem linearen Gleichungssystem pro Iterationsschritt:

$$\begin{bmatrix} \Delta_{\mathcal{M}_1} \\ \Delta_{\mathcal{A}_1} \\ \vdots \\ \Delta_{\mathcal{M}_n} \\ \Delta_{\mathcal{A}_n} \end{bmatrix} = \mathbf{J} \begin{bmatrix} \Delta e_1 \\ \Delta f_1 \\ \vdots \\ \Delta e_n \\ \Delta f_n \end{bmatrix} \quad (2.45)$$

Die Anordnung der Gleichungen erfolgt dabei so, dass die Imaginärteil-Gleichungen ($\Delta_{\mathcal{M}_k}$) vor den Realteil-Gleichungen ($\Delta_{\mathcal{A}_k}$) stehen. Diese Sortierung bewirkt, dass die Jacobi-Matrix $\mathbf{J}^*$ diagonaldominant wird, da die Suszeptanzen B_{km} betragsmäßig typischerweise deutlich größer als die Konduktanzen G_{km} sind [4].

Die Jacobi-Matrix $\mathbf{J}^*$ besitzt die gleiche Blockstruktur wie die $(2n \times 2n)$ -Knotenadmittanzmatrix, wobei jeder Netzwerkzweig durch einen (2×2) -Block repräsentiert wird. Für die

Außerdiagonalelemente gilt bei Verbindung zwischen PQ-Knoten k und m :

$$\mathbf{J}_{km} = \begin{bmatrix} -B_{km} & G_{km} \\ G_{km} & B_{km} \end{bmatrix} \quad (2.46)$$

Diese Blöcke sind identisch mit den entsprechenden Elementen der Knotenadmittanzmatrix in kartesischer Darstellung und bleiben daher während des gesamten Iterationsprozesses **konstant**. Dies ist der zentrale Vorteil der CIM gegenüber der konventionellen Formulierung: Die aufwendige Neuberechnung der Außerdiagonalelemente entfällt vollständig.

Die (2×2) -Diagonalblöcke für einen PQ-Knoten k haben die Form:

$$\mathbf{J}_{kk} = \begin{bmatrix} -B'_{kk} & G'_{kk} \\ G''_{kk} & B''_{kk} \end{bmatrix} \quad (2.47)$$

mit den modifizierten Elementen [4]:

$$B'_{kk} = B_{kk} - a_k \quad (2.48)$$

$$G'_{kk} = G_{kk} - b_k \quad (2.49)$$

$$G''_{kk} = G_{kk} - c_k \quad (2.50)$$

$$B''_{kk} = -B_{kk} - d_k \quad (2.51)$$

Die Terme a_k , b_k , c_k und d_k hängen von der spezifizierten Last und Erzeugung am Knoten k sowie vom gewählten Lastmodell ab. Für den Fall einer reinen Konstantleistungslast ($\alpha_P = 1$, $\alpha_Z = \alpha_I = 0$) vereinfachen sich diese zu [4]:

$$a_k = -d_k = \frac{Q'_k(e_k^2 - f_k^2) - 2e_k f_k P'_k}{V_k^4} \quad (2.52)$$

$$b_k = -c_k = \frac{P'_k e_k^2 - f_k^2 + 2e_k f_k Q'_k}{V_k^4} \quad (2.53)$$

mit

$$P'_k = P_{G(k)} - P_{0k} \alpha_P \quad (2.54)$$

$$Q'_k = Q_{G(k)} - Q_{0k} \alpha_P \quad (2.55)$$

wobei $P_{G(k)}$ und $Q_{G(k)}$ die erzeugte Wirk- bzw. Blindleistung am Knoten k bezeichnen und P_{0k} , Q_{0k} die Nennlastwerte sind.

Da diese Terme von der aktuellen Spannung V_k abhängen, müssen die Diagonalblöcke in jedem Iterationsschritt aktualisiert werden. Für Knoten ohne spezifizierte Last oder mit reiner Impedanzlast ($a_p = 1$, $b_p = c_p = 0$) reduzieren sich die Terme a_k , b_k ,

c_k und d_k auf konstante Admittanzanteile, sodass der Diagonalblock während des gesamten Iterationsprozesses unverändert bleibt [4].

Die Spannungskorrekturen werden nach der Lösung des linearen Gleichungssystems in kartesischen Koordinaten berechnet und können anschließend in polare Koordinaten umgerechnet werden:

$$\Delta\theta_k = \arctan\left(\frac{e_k \Delta f_k - f_k \Delta e_k}{e_k^2 + f_k^2}\right) \quad (2.56)$$

$$\Delta V_k = \frac{e_k \Delta e_k + f_k \Delta f_k}{V_k} \quad (2.57)$$

Die Aktualisierung der Zustandsgrößen erfolgt gemäß:

$$\theta_k^{(h+1)} = \theta_k^{(h)} + \Delta\theta_k^{(h+1)} \quad (2.58)$$

$$V_k^{(h+1)} = V_k^{(h)} + \Delta V_k^{(h+1)} \quad (2.59)$$

wobei h den Iterationszähler bezeichnet. Es ist hervorzuheben, dass die Konvergenztrajektorie der CIM identisch mit der des konventionellen Newton-Raphson-Verfahrens in Polarkoordinaten ist [4].

2.3.3 Behandlung von PV-Knoten in der CIM

An PV-Knoten (Generatorknoten) sind die Wirkleistungseinspeisung P_k^{sp} und der Spannungsbetrag V_k^{sp} vorgegeben, während die Blindleistung Q_k eine unbekannte (abhängige) Variable darstellt. Dies erfordert eine besondere Behandlung innerhalb der CIM, da der Blindleistungs-Mismatch ΔQ_k in den Gleichungen (2.41) und (2.42) nicht direkt bekannt ist.

Der in [4] vorgeschlagene Ansatz löst dieses Problem elegant durch die Einführung einer zusätzlichen abhängigen Variablen ΔQ_k für jeden PV-Knoten sowie einer zusätzlichen linearisierten Gleichung, die die Bedingung eines konstanten Spannungsbetrags $|\Delta V_k| = 0$ erzwingt. Die linearisierte Form dieser Bedingung lautet:

$$e_k \Delta e_k + f_k \Delta f_k = 0 \quad (2.60)$$

Diese Gleichung kann umgestellt werden zu:

$$\Delta f_k = -\frac{e_k}{f_k} \Delta e_k \quad (2.61)$$

Durch Einsetzen von Gleichung (2.61) in die Strommismatch-Gleichungen des Knotens k wird die abhängige Variable ΔQ_k (in Form von $\Delta Q_k/V_k^2$) anstelle von Δf_k als Unbekannte im Gleichungssystem verwendet. Dies bedeutet, dass im Lösungsvektor

die Komponente Δf_k für jeden PV-Knoten durch $\Delta Q_k/V_k^2$ ersetzt wird.

Das resultierende erweiterte Gleichungssystem für einen PV-Knoten k lässt sich in Blockform schreiben als:

$$\begin{bmatrix} \Delta M_k \\ \Delta \Pi_k \\ 0 \end{bmatrix} = \begin{bmatrix} -B_{kk} & G_{kk} & f_k/V_k^2 \\ G_{kk} & B_{kk} & -e_k/V_k^2 \\ e_k & f_k & 0 \end{bmatrix} \begin{bmatrix} \Delta e_k \\ \Delta f_k \\ \Delta Q_k \end{bmatrix} \quad (2.62)$$

Durch Elimination der dritten Zeile und Substitution von Δf_k ergibt sich ein (2×2) -Diagonalblock für den PV-Knoten k :

$$\mathbf{J}_{kk,\text{PV}} = \begin{bmatrix} -B_{kk} - G_{kk} \frac{e_k}{f_k} & \frac{f_k}{V_k^2} \\ G_{kk} + B_{kk} \frac{e_k}{f_k} & -\frac{e_k}{V_k^2} \end{bmatrix} \quad (2.63)$$

Die außerdiagonalen (2×2) -Blöcke, die einen PV-Knoten k mit einem benachbarten Knoten l verbinden, verändern sich ebenfalls. Die erste Spalte bleibt identisch mit der Admittanzmatrix, während die zweite Spalte zu Null wird, da Δf_k durch die Spannungsbedingung eliminiert wurde:

$$\mathbf{J}_{lk,\text{PV}} = \begin{bmatrix} -B_{lk} - G_{lk} \frac{e_k}{f_k} & 0 \\ G_{lk} + B_{lk} \frac{e_k}{f_k} & 0 \end{bmatrix} \quad (2.64)$$

Diese Implementierung bewahrt die $(2n \times 2n)$ -Struktur der Jacobi-Matrix unabhängig von der Anzahl der PV-Knoten im System. Die Unbekannte Δf_k wird durch ΔQ_k ersetzt, sodass nach der Lösung des Gleichungssystems der aktualisierte Blindleistungs-Mismatch direkt verfügbar ist. Die Spannung am PV-Knoten wird ausschließlich über die Winkelkorrektur aktualisiert:

$$\theta_k^{(h+1)} = \theta_k^{(h)} + \Delta \theta_k^{(h+1)} \quad (2.65)$$

während der Spannungsbetrag $V_k = V_k^{\text{sp}}$ konstant bleibt.

Abbildung ?? illustriert schematisch die Struktur der Jacobi-Matrix $\mathbf{J}^*$ für ein System mit PQ- und PV-Knoten. Die mit einem Stern (*) markierten Elemente werden in jeder Iteration aktualisiert, während alle übrigen Elemente konstant bleiben.

Ein wesentlicher Vorteil dieser Formulierung ist, dass die Konvergenzcharakteristik vollständig mit der des konventionellen Newton-Raphson-Verfahrens in Polarkoordinaten übereinstimmt. Dies wurde von da Costa et al. [4] anhand praxisrelevanter Testnetze mit bis zu 2111 Knoten nachgewiesen, wobei identische Konvergenzverläufe bei einer gleichzeitigen Reduktion der Rechenzeit um über 20% erzielt wurden.

Die Rechenzeiteinsparung resultiert primär aus der Konstanz der außerdiagonalen Jacobi-Elemente sowie dem Entfall trigonometrischer Funktionsauswertungen.

2.3.4 Q-Limits und PV-zu-PQ-Umwandlung

In realen Energieversorgungssystemen unterliegen Generatoren physikalischen Blindleistungsgrenzen:

$$Q_k^{\min} \leq Q_k \leq Q_k^{\max} \quad (2.66)$$

Solange die am PV-Knoten berechnete Blindleistung Q_k innerhalb dieser Grenzen liegt, kann der Generator die spezifizierte Spannung aufrechterhalten. Wird eine der Grenzen verletzt, muss der PV-Knoten in einen PQ-Knoten umgewandelt werden, wobei Q_k^{sp} auf den verletzten Grenzwert fixiert wird.

Bei der Umwandlung ändert sich die Struktur der Jacobi-Matrix lokal:

- Der (2×2) -Diagonalblock wechselt von der PV-Form (Gleichung (2.63)) zur PQ-Form (Gleichung (2.47)), wobei Q_k^{sp} den fixierten Grenzwert annimmt.
- Die außerdiagonalen Blöcke in der Spalte des betroffenen Knotens kehren zur Standardform gemäß Gleichung (2.46) zurück.
- Die Unbekannte ΔQ_k wird durch ΔV_{mk} ersetzt, sodass der Spannungsbetrag frei variieren kann.

Die Rückumwandlung in einen PV-Knoten erfolgt, wenn die berechnete Spannung den Sollwert V_k^{sp} überschreitet (bei Fixierung auf $Q_k^{\min}$) bzw. unterschreitet (bei Fixierung auf $Q_k^{\max}$). Um Oszillationen zwischen den Knotentypen zu vermeiden, empfiehlt es sich, die Umwandlungslogik erst nach einigen Anfangsiterationen zu aktivieren.

Dieser Wechsel ist innerhalb der CIM besonders effizient, da lediglich lokale Modifikationen an einzelnen (2×2) -Blöcken erforderlich sind und die $(2n \times 2n)$ -Gesamtstruktur sowie die Sparse-Faktorisierungsreihenfolge unverändert bleiben.

2.4 Fixed-Point-Iteration und Konvergenz

Die Fixed-Point-Iteration (FPI), auch als *Successive Approximation Method* (SAM) bezeichnet, bietet eine Alternative zum Newton-Raphson-Verfahren, die sich durch ihre einfache Formulierung, geringe Kosten pro Iteration und natürliche Eignung zur Parallelisierung auszeichnet. Die Methode wurde von Giraldo et al. [2] im Rahmen der Current Injection Method unter Verwendung der Laurent-Reihenentwicklung vorgeschlagen und bildet die Grundlage des in Kapitel 3 vorgestellten Tensor Power Flow.

2.4.1 Laurent-Reihen-Approximation

Die Nichtlinearität der Lastflussgleichungen (??) resultiert aus dem Term $(V_k^\phi)^{-1}$, der im spezifizierten Strom (2.5) auftritt. Giraldo et al. [2] schlagen eine Linearisierung mittels Laurent-Reihenentwicklung vor.

Proposition 2.1 (Laurent-Approximation, [2, Proposition 1]). *Die Nichtlinearität $f(V_i) = (V_i)^{-1}$ an einem Knoten $i \in \Omega_d$ wird um einen Arbeitspunkt $V^0 \in \mathbb{C}$ mittels Laurent-Reihe approximiert als:*

$$f(V_i) \approx \frac{2}{V^0} - \frac{V_i}{(V^0)^2} \quad (2.67)$$

unter der Annahme $V_i = V^0 - \Delta V$ mit $\|\Delta V\|/\|V^0\| < 1$.

Die Annahme $\|\Delta V\|/\|V^0\| < 1$ ist physikalisch gerechtfertigt, da Knotenspannungen in Verteilnetzen im Normalbetrieb innerhalb eines engen Bandes um den Nennwert liegen (typisch 0,9 bis 1,1 p.u.) [2]. Im iterativen Algorithmus wird der Arbeitspunkt V^0 nach jeder Iteration auf den aktuellen Spannungswert aktualisiert, sodass die Approximation mit jeder Iteration genauer wird.

2.4.2 Herleitung des FPI-Algorithmus

Ausgangspunkt ist die Knotengleichung für die Lastknoten aus (2.1):

$$-\mathbf{I}_d = Y_{ds}\mathbf{V}_s + Y_{dd}\mathbf{V}_d \quad (2.68)$$

Der Strom an den Lastknoten ergibt sich unter Berücksichtigung des ZIP-Lastmodells (vgl. Abschnitt 2.1.4) nach Giraldo et al. [2] zu:

$$\mathbf{I}_d = \left(\alpha_P \text{diag}(\mathbf{V}_d^*)^{-1} + \text{diag}(\alpha_I) + \alpha_Z \text{diag}(\mathbf{V}_d) \right) \mathbf{S}_n^* \quad (2.69)$$

Durch Anwendung der Laurent-Approximation (Theorem 2.1) auf den nichtlinearen Term $\text{diag}(\mathbf{V}_d^*)^{-1}$ und Einsetzen in (2.68) ergibt sich das folgende lineare Gleichungssystem [2]:

$$\mathbf{A}\mathbf{V}_d^* - \mathbf{B}\mathbf{V}_d = \mathbf{C} + \mathbf{D} \quad (2.70)$$

mit den Matrizen und Vektoren:

$$\mathbf{A} = \text{diag}(\alpha_P \odot V^{0*-2} \odot \mathbf{S}_n^*) \quad (2.71)$$

$$\mathbf{B} = \text{diag}(\alpha_Z \odot \mathbf{S}_n^*) + Y_{dd} \quad (2.72)$$

$$\mathbf{C} = Y_{ds}\mathbf{V}_s + \alpha_I \odot \mathbf{S}_n^* \quad (2.73)$$

$$\mathbf{D} = 2\alpha_P \odot V^{0*-1} \odot \mathbf{S}_n^* \quad (2.74)$$

Hierbei bezeichnen $\odot$ das Hadamard-Produkt, V^{0*-1} den element-weisen Kehrwert des konjugierten Arbeitspunktvektors, und V^{0*-2} das element-weise Quadrat davon. Die Matrix $\mathbf{B} \in \mathbb{C}^{b\phi \times b\phi}$ und der Vektor $\mathbf{C} \in \mathbb{C}^{b\phi \times 1}$ sind innerhalb des iterativen Prozesses *konstant*, da sie nur von der Netztopologie (Y_{dd} , Y_{ds}), der Slack-Spannung ($\mathbf{V}_s$) und den spannungsunabhängigen Lastanteilen abhängen. Lediglich $\mathbf{A}$ und $\mathbf{D}$ müssen in jeder Iteration aktualisiert werden, da sie vom Arbeitspunkt V^0 abhängen [2].

2.4.3 Die Successive Approximation Method (SAM)

Giraldo et al. [2] schlagen vor, das Gleichungssystem (2.70) nach $\mathbf{V}_d$ umzustellen:

$$\mathbf{V}_d^{(k+1)} = \mathbf{B}^{-1} \left(\mathbf{A}^{(k)} \mathbf{V}_d^{*(k)} - \mathbf{C} - \mathbf{D}^{(k)} \right) \quad (2.75)$$

Diese Gleichung hat die Struktur einer Fixpunktiteration $\mathbf{V}_d^{(k+1)} = f(\mathbf{V}_d^{(k)})$ und kann sukzessiv gelöst werden. Der Algorithmus ist in Algorithmus 1 dargestellt.

Algorithm 1 Successive Approximation Method (SAM) nach [2]

Require: $\mathbf{S}_n$, Y_{dd} , Y_{ds} , $\mathbf{V}_s$, Toleranz ε , max. Iterationen K

Ensure: $\mathbf{V}_d$

- 1: Initialisiere $k = 0$, $\text{tol} = \infty$
 - 2: Initialisiere $V_{i,\phi}^0 = [1, a^2, a]$ für alle $i \in \Omega_d$ {Flat Start, $a = e^{j2\pi/3}$ }
 - 3: Berechne $\mathbf{B}^{-1}$ und $\mathbf{C}$ gemäß (2.72)–(2.73) {Einmalig!}
 - 4: **while** $\text{tol} \geq \varepsilon$ **und** $k < K$ **do**
 - 5: Berechne $\mathbf{A}^{(k)}$ und $\mathbf{D}^{(k)}$ gemäß (2.71)–(2.74)
 - 6: $\mathbf{V}_d^{(k+1)} = \mathbf{B}^{-1} \left(\mathbf{A}^{(k)} \mathbf{V}_d^{*(k)} - \mathbf{C} - \mathbf{D}^{(k)} \right)$
 - 7: Berechne Leistungsmismatch: $\mathbf{S}_d^{*(k+1)} = \text{diag}(\mathbf{V}_d^{*(k+1)})(Y_{ds} \mathbf{V}_s + Y_{dd} \mathbf{V}_d^{(k+1)})$
 - 8: $\text{tol} \leftarrow \max \|\mathbf{S}_d^* - \mathbf{S}_d^{*(k+1)}\|$
 - 9: $\mathbf{V}_d^0 \leftarrow \mathbf{V}_d^{(k+1)}$ {Arbeitspunkt aktualisieren}
 - 10: $k \leftarrow k + 1$
 - 11: **end while**
 - 12: Berechne $\mathbf{S}_s^* = \text{diag}(\mathbf{V}_s^*)(\mathbf{Y}_{ss} \mathbf{V}_s + \mathbf{Y}_{sd} \mathbf{V}_d^{(k)})$
 - 13: **return** $\mathbf{V}_d^{(k)}$, $\mathbf{S}_s$
-

Die zentrale rechentechnische Eigenschaft des SAM-Algorithmus ist, dass die Matrix $\mathbf{B}^{-1}$ nur *einmal* berechnet werden muss und in allen Iterationen wiederverwendet wird. In jeder Iteration wird lediglich eine Matrix-Vektor-Multiplikation ($\mathbf{B}^{-1} \cdot (\dots)$) durchgeführt [2]. Dies steht im Gegensatz zum Newton-Raphson-Verfahren und zur Direct Solution (DS) aus [2], bei denen in jeder Iteration ein vollständiges lineares Gleichungssystem gelöst werden muss.

Für große Netze kann die direkte Berechnung von $\mathbf{B}^{-1}$ speicherintensiv sein, da die Inverse einer dünnbesetzten Matrix im Allgemeinen vollbesetzt ist. Giraldo et al. [2]

verweisen auf effiziente Algorithmen zum Aufbau der Z-Bus-Matrix [Davis2006] sowie auf die Möglichkeit, $\mathbf{B}$ stattdessen zu faktorisieren (z. B. LDU-Zerlegung) und die Faktorisierung wiederzuverwenden.

2.4.4 Konvergenzbeweis mittels Banach-Fixpunktsatz

Die Konvergenz des SAM-Algorithmus wird von Giraldo et al. [2] über den Banach'schen Fixpunktsatz nachgewiesen. Dieser Satz liefert nicht nur die Existenz und Eindeutigkeit des Fixpunktes, sondern auch die Unabhängigkeit vom Startwert.

Satz 2.2 (Banach'scher Fixpunktsatz). *Sei (X, d) ein vollständiger metrischer Raum und $T : X \rightarrow X$ eine Kontraktion, d. h. es existiert ein $\eta \in [0, 1)$ mit*

$$d(T(\mathbf{x}), T(\mathbf{y})) \leq \eta \cdot d(\mathbf{x}, \mathbf{y}) \quad \forall \mathbf{x}, \mathbf{y} \in X. \quad (2.76)$$

Dann besitzt T genau einen Fixpunkt $\mathbf{x}^ = T(\mathbf{x}^*)$, und die Folge $\mathbf{x}^{(k+1)} = T(\mathbf{x}^{(k)})$ konvergiert für jeden Startwert $\mathbf{x}^{(0)} \in X$ gegen $\mathbf{x}^*$.*

Proposition 2.3 (Kontraktion der SAM, [2, Proposition 2]). *Der SAM-Algorithmus (Algorithmus 1) ist eine Kontraktionsabbildung der Form $\mathbf{V}_d^{(k+1)} = f(\mathbf{V}_d^{(k)})$ und besitzt eine eindeutige, vom Startwert unabhängige Lösung, falls:*

$$\|f(\mathbf{V}_d^{(k)}) - f(\mathbf{U}_d)\| \leq \eta \cdot \|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| \quad (2.77)$$

mit dem Kontraktionsfaktor:

$$\eta = \max_{i \in \Omega_d} \left\{ \frac{\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\|}{v^2} \right\}, \quad 0 \leq \eta < 1 \quad (2.78)$$

wobei $\mathbf{U}_d$ die exakte Lösung ist und $v = \min_{i \in \Omega_d} \|u_i\|$ die minimale Spannungsmagnitude im Netz.

Beweis. Basierend auf dem Banach-Fixpunktsatz existiert der Fixpunkt $\mathbf{U}_d$ mit $f(\mathbf{U}_d) = \mathbf{U}_d$ und ist eindeutig, wenn $f(\cdot)$ eine Kontraktion auf $\mathbf{V}_d$ ist. Die Differenz zweier aufeinanderfolgender Iterationen ergibt sich nach [2] zu:

$$\|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| = \|f(\mathbf{V}_d^{(k)}) - f(\mathbf{U}_d)\| = \|\mathbf{B}^{-1} \mathbf{A}^{(k)} (\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*)\| \quad (2.79)$$

Da $\mathbf{A}^{(k)}$ eine Diagonalmatrix ist, kann abgeschätzt werden:

$$\|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| \leq \frac{\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\|}{\|\mathbf{V}_d^{*(k)}\|^2} \cdot \|\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*\| \leq \eta \cdot \|\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*\| \quad (2.80)$$

Damit $\eta < 1$ gilt, müssen zwei Bedingungen erfüllt sein [2]:

1. $v > 0$ (Spannungsbeträge sind im Normalbetrieb stets positiv)
2. $\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\| < v^2$

□

2.4.5 Physikalische Interpretation der Konvergenzbedingung

Die abstrakte Bedingung $\eta < 1$ kann physikalisch interpretiert werden. Unter der Annahme rein leistungskonstanter Last ($\alpha_P = 1$) wird die Matrix $\mathbf{B} = \mathbf{Y}_{dd}$ und somit $\mathbf{B}^{-1} = \mathbf{Z}_B$ die Bus-Impedanzmatrix (Annahme 2 in [2]). Definiert man die äquivalente Lastimpedanz am Knoten i als:

$$Z_{L,i} = \frac{v^2}{\|S_{n,i}^*\|} \quad (2.81)$$

und nutzt die Eigenschaft, dass die Diagonalelemente der Bus-Impedanzmatrix Z_{ii} die Thévenin-Impedanzen darstellen (mit $\|Z_{ii}\| \geq \|Z_{ij}\|$ für $i \neq j$), so ergibt sich der Kontraktionsfaktor näherungsweise zu [2]:

$$\eta \approx \max_{i \in \Omega_d} \left\{ \frac{\|Z_{ii}\|}{\|Z_{L,i}\|} \right\} \quad (2.82)$$

Die Bedingung $\eta < 1$ ist somit äquivalent zu:

$$\|Z_{ii}\| < \|Z_{L,i}\| \quad \forall i \in \Omega_d \quad (2.83)$$

Diese Ungleichung besagt: *Die Thévenin-Impedanz an jedem Knoten muss betragsmäßig kleiner sein als die äquivalente Lastimpedanz.* Nach dem Theorem der maximalen Leistungsübertragung [3] sind beide Impedanzen genau am Punkt maximaler Leistungsübertragung (*maximum loading point*, MLP) gleich groß [2]. In diesem Grenzfall gilt $\eta = 1$ und die Konvergenz ist nicht mehr gewährleistet.

Daraus folgen zwei zentrale Aussagen:

1. Die Bedingung (2.83) ist für den *normalen Betriebspunkt* eines Netzes stets erfüllt, da dort $\|Z_{L,i}\| \gg \|Z_{ii}\|$ gilt (hohe Spannung, niedriger Strom). Der SAM-Algorithmus konvergiert somit stets zur operationellen Lösung, unabhängig vom Startwert.
2. Die Konvergenzrate verschlechtert sich mit zunehmender Last ($\eta \rightarrow 1$ für $\lambda \rightarrow \lambda_{\text{mlp}}$) und mit steigendem Impedanzbetrag der Leitungen ($\|Z_{ii}\|$ wächst mit dem R/X -Verhältnis).

Diese physikalische Interpretation wurde auch in [1] aufgegriffen und dort durch eine geometrische Herleitung im Impedanzraum ergänzt (vgl. ??).

Kapitel 3

Der Tensor Power Flow

Aufbauend auf den in Kapitel 2 eingeführten Grundlagen der Fixed-Point-Iteration wird in diesem Kapitel der Tensor Power Flow (TPF) nach Salazar Duque et al. [1] vorgestellt. Der TPF erweitert den eindimensionalen FPI-Algorithmus zu einer mehrdimensionalen Formulierung, die Hunderttausende von Lastflüssen gleichzeitig berechnen kann. Dabei werden zwei Formulierungen präsentiert sowie deren Eignung für CPU- und GPU-Parallelisierung diskutiert. Das Kapitel schließt mit einer Analyse der fundamentalen Limitation des TPF: der fehlenden Unterstützung von PV-Knoten.

3.1 Grundalgorithmus

Der TPF basiert auf dem in Abschnitt 2.4 eingeführten FPI-Algorithmus nach Giraldo et al. [2]. Ausgangspunkt ist das Netzmodell aus (2.1) mit einem Einspeiseknoten, $b = |\Omega_d|$ Lastknoten und $\phi = |\Omega_\phi|$ Phasen. Die Iterationsvorschrift lautet [1]:

$$\mathbf{v}_{(n+1)} = \mathbf{F} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{w} \quad (3.1)$$

wobei $\mathbf{v}_{(n)}^{*\circ(-1)} \in \mathbb{C}^{b\phi \times 1}$ den element-weisen Kehrwert des konjugierten Spannungsvektors der n -ten Iteration bezeichnet. Die Operation $(\cdot)^{*\circ(-1)}$ ist definiert als:

$$\left(\mathbf{v}^{*\circ(-1)}\right)_i = \frac{1}{v_i^*} \quad \forall i = 1, \dots, b\phi \quad (3.2)$$

Die konstanten Hilfsmatrizen und -vektoren sind definiert als [1]:

$$\mathbf{A} = \text{diag}(\alpha_P \odot \mathbf{s}^*) \quad (3.3)$$

$$\mathbf{B} = \text{diag}(\alpha_Z \odot \mathbf{s}^*) + Y_{dd} \quad (3.4)$$

$$\mathbf{c} = Y_{ds}\mathbf{v}_s + \alpha_I \odot \mathbf{s}^* \quad (3.5)$$

$$\mathbf{F} = -\mathbf{B}^{-1}\mathbf{A} \quad (3.6)$$

$$\mathbf{w} = -\mathbf{B}^{-1}\mathbf{c} \quad (3.7)$$

Dabei bezeichnen α_P , α_I und α_Z die Koeffizienten des ZIP-Lastmodells (vgl. Abschnitt 2.1.4), $\mathbf{s} \in \mathbb{C}^{b\phi \times 1}$ den Vektor der komplexen Knotenleistungen und $\odot$ das Hadamard-Produkt. Die Matrizen $\mathbf{A} \in \mathbb{C}^{b\phi \times b\phi}$, $\mathbf{B} \in \mathbb{C}^{b\phi \times b\phi}$ und der Vektor $\mathbf{c} \in \mathbb{C}^{b\phi \times 1}$ sind für einen gegebenen Betriebspunkt *konstant* innerhalb des iterativen Prozesses. Folglich können $\mathbf{F} \in \mathbb{C}^{b\phi \times b\phi}$ und $\mathbf{w} \in \mathbb{C}^{b\phi \times 1}$ einmal vorausberechnet und in jeder Iteration wiederverwendet werden.

Für den in Verteilnetzen häufigen Spezialfall rein konstanter Leistung ($\alpha_P = 1$, $\alpha_I = \alpha_Z = 0$) vereinfachen sich die Ausdrücke zu:

$$\mathbf{B} = Y_{dd} \quad (3.8)$$

$$\mathbf{F} = -Y_{dd}^{-1} \text{diag}(\mathbf{s}^*) = -Z_B \text{diag}(\mathbf{s}^*) \quad (3.9)$$

$$\mathbf{w} = -Z_B Y_{ds} \mathbf{v}_s \quad (3.10)$$

wobei $Z_B = Y_{dd}^{-1}$ die Bus-Impedanzmatrix bezeichnet. Die Iteration (3.1) nimmt dann die kompakte Form an:

$$\mathbf{v}_{(n+1)} = Z_B \cdot \left(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*\circ(-1)} \right) + \mathbf{w} \quad (3.11)$$

Der Term $\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*\circ(-1)} = \mathbf{s}^* \oslash \mathbf{v}_{(n)}^*$ (mit $\oslash$ als element-weiser Division) entspricht physikalisch dem konjugierten Stromvektor $\mathbf{i}^* = \mathbf{s}^* / \mathbf{v}^*$, wobei die Beziehung $s_k = v_k \cdot i_k^*$ aus (2.4) genutzt wurde. Die Multiplikation $Z_B \cdot \mathbf{i}^*$ liefert den Spannungsabfall über die Netzipedanzen, und $\mathbf{w}$ korrigiert für den Einfluss der festen Slack-Spannung $\mathbf{v}_s$.

3.2 DENSE- UND SPARSE-FORMULIERUNG

Die zentrale Innovation des TPF besteht in der Erweiterung des eindimensionalen FPI-Algorithmus (3.1) zu einer mehrdimensionalen (tensorisierten) Formulierung, die eine große Anzahl τ von Lastflüssen gleichzeitig verarbeiten kann.

3.2.1 Motivation: Der Tensor der Lastdaten

In typischen Anwendungen wie Zeitreihensimulationen, probabilistischen Lastflüssen oder maschinellem Lernen liegen die Lastdaten als mehrdimensionaler Tensor vor:

$$\mathcal{S} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi} \quad (3.12)$$

wobei die Dimensionen beispielsweise Experimente (p), Szenarien (r) und Zeitschritte (t) repräsentieren können. Um die Berechnung effizient zu gestalten, werden alle nicht-physikalischen Dimensionen zu einem einzigen Index $\tau = \dots \times p \times r \times t$ zusammengefasst (*Reshape-Operation*). Die resultierenden zweidimensionalen Matrizen werden mit der Punkt-Notation gekennzeichnet:

$$\mathcal{S} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi} \xrightarrow{\text{reshape}} \dot{S} \in \mathbb{C}^{b\phi \times \tau} \quad (3.13)$$

Jede Spalte von $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$ enthält den Leistungsvektor eines einzelnen Lastflusses, und die τ Spalten repräsentieren alle zu berechnenden Szenarien. Analog werden die Spannungsmatrizen $\dot{V}_{(n+1)}, \dot{V}_{(n)}^{*o(-1)} \in \mathbb{C}^{b\phi \times \tau}$ definiert.

3.2.2 Dense-Formulierung

Die tensorisierte FPI in ihrer dense Formulierung lautet [1]:

$$\mathcal{V}_{(n+1)} = \mathcal{F} \cdot \mathcal{V}_{(n)}^{*o(-1)} + \mathcal{W} \quad (3.14)$$

mit den Tensor-Dimensionen $\mathcal{F} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi \times b\phi}$ und $\mathcal{V}_{(n+1)}, \mathcal{V}_{(n)}^{*o(-1)}, \mathcal{W} \in \mathbb{C}^{\dots \times p \times r \times b\phi \times t}$. Im Spezialfall konstanter Leistung und fester Netztopologie besteht der Tensor $\mathcal{F}$ lediglich aus Wiederholungen der identischen Matrix Z_B . Ebenso ist $\mathcal{W}$ eine Wiederholung des konstanten Vektors $\mathbf{w}$. Durch den Reshape zu zweidimensionalen Matrizen ergibt sich die effiziente Berechnungsvorschrift aus Algorithm 1 in [1]:

$$\dot{V}_{(n+1)}[:, i] = Z_B \left(\dot{S}^*[:, i] \odot \dot{V}_{(n)}^{*o(-1)}[:, i] \right)^* + \mathbf{w} \quad \forall i = 1, \dots, \tau \quad (3.15)$$

wobei $[:, i]$ die i -te Spalte der jeweiligen Matrix bezeichnet. Diese τ Operationen können als eine einzige Matrix-Matrix-Multiplikation zusammengefasst werden:

$$\dot{V}_{(n+1)} = Z_B \cdot \underbrace{\left(\dot{S}^* \odot \dot{V}_{(n)}^{*o(-1)} \right)}_{\in \mathbb{C}^{b\phi \times \tau}} + \dot{W} \quad (3.16)$$

wobei $\dot{W} = [\mathbf{w}, \mathbf{w}, \dots, \mathbf{w}] \in \mathbb{C}^{b\phi \times \tau}$ den τ -fach duplizierten Konstantvektor enthält und die Operationen $\odot$ und $(\cdot)^{*o(-1)}$ element-weise auf die gesamten Matrizen angewendet werden.

Gleichung (3.16) stellt eine Multiplikation einer dichten Matrix $Z_B \in \mathbb{C}^{b\phi \times b\phi}$ mit einer Matrix $\in \mathbb{C}^{b\phi \times \tau}$ dar. Dies ist die Schlüsseloperation des TPF: Alle τ Lastflüsse werden durch *eine einzige* Matrix-Matrix-Multiplikation gleichzeitig berechnet.

Algorithm 2 Tensor Power Flow – Dense-Formulierung nach [1]

Require: $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$, $Z_B \in \mathbb{C}^{b\phi \times b\phi}$, $\mathbf{w} \in \mathbb{C}^{b\phi \times 1}$, Toleranz ε , max. Iterationen K

Ensure: $\dot{V}_n \in \mathbb{C}^{b\phi \times \tau}$, Iterationszahl n

```

1:  $\dot{V}_0 = \mathbf{1} + \mathbf{j}\mathbf{0} \in \mathbb{C}^{b\phi \times \tau}$ ;  $n = 0$ ;  $\text{tol} = \infty$ 
2: while  $\text{tol} \geq \varepsilon$  und  $n < K$  do
3:   for  $i = 1$  bis  $\tau$  do
4:      $\dot{V}_{(n+1)}[:, i] = Z_B \left( \dot{S}[:, i] \odot \dot{V}_{(n)}^{*o(-1)}[:, i] \right)^* + \mathbf{w}$ 
5:   end for
6:    $\text{tol} = \max \left( \|s_{(n+1)} - s_{(n)}\|_2 \right)$ ;  $n = n + 1$ 
7: end while
8: return  $\dot{V}_{(n)}$ ,  $n$ 
```

Es ist anzumerken, dass der Reshape von (3.14) zu (3.16) die Konvergenz des FPI-Algorithmus nicht beeinflusst, da die Aktualisierung der Spannungswerte identisch zu (3.1) bleibt – lediglich τ Instanzen werden parallel ausgeführt [1].

3.2.3 Sparse-Formulierung

Ein Nachteil der dense Formulierung ist, dass die Matrix $Z_B = Y_{dd}^{-1}$ vollbesetzt (dense) ist, obwohl die Admittanzmatrix Y_{dd} dünnbesetzt (sparse) ist. Für große Netze ($b\phi > 1000$) kann die Speicherung von Z_B problematisch werden. Salazar Duque et al. [1] schlagen daher eine sparse Formulierung vor, die die Inversion von Y_{dd} vermeidet.

Die Iteration (3.1) wird umformuliert zu:

$$\mathcal{M} \cdot \mathcal{V}_{(n+1)} = \mathcal{V}_{(n)}^{*o(-1)} + \mathcal{H} \quad (3.17)$$

wobei:

$$\mathcal{M} = -\mathbf{A}^{o(-1)} \mathbf{B} \quad (3.18)$$

$$\mathcal{H} = \mathbf{A}^{o(-1)} \mathbf{C} \quad (3.19)$$

mit $\mathbf{A}$, $\mathbf{B}$ und $\mathbf{C}$ gemäß (3.3)–(3.5). Da $\mathbf{A}$ eine Diagonalmatrix ist, überträgt sich die Dünnbesetztheit von Y_{dd} auf $\mathcal{M}$.

Die Reshaped-Form von (3.17) ergibt ein lineares Gleichungssystem der Struktur $\mathbf{A}\mathbf{x} = \mathbf{b}$:

$$\underbrace{\dot{M}}_{\mathbf{A}} \cdot \underbrace{\dot{V}_{(n+1)}}_{\mathbf{x}} = \underbrace{\dot{V}_{(n)}^{*o(-1)} + \dot{H}}_{\mathbf{b}} \quad (3.20)$$

Die Matrix $\dot{M} \in \mathbb{C}^{(\tau \cdot b\phi) \times (\tau \cdot b\phi)}$ ist die Block-Diagonalverkettung der Submatrizen:

$$\dot{M} = \begin{bmatrix} \mathbf{M}_1 & 0 & \cdots & 0 \\ 0 & \mathbf{M}_2 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{M}_\tau \end{bmatrix} \quad (3.21)$$

Die Vektoren $\dot{V}_{(n+1)}$, $\dot{V}_{(n)}^{*\circ(-1)}$ und $\dot{H}$ werden vertikal angeordnet. Das System (3.20) wird iterativ mittels eines sparse Direktl6sers berechnet.

Algorithm 3 Tensor Power Flow – Sparse-Formulierung nach [1]

Require: $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$, $Y_{dd} \in \mathbb{C}^{b\phi \times b\phi}$ (sparse), $Y_{ds} \in \mathbb{C}^{b\phi \times \phi}$ (sparse), Toleranz ε , max. Iterationen K

Ensure: $\dot{V}_n \in \mathbb{C}^{b\phi \times \tau}$, Iterationszahl n

- 1: Berechne $\dot{M}$ und $\dot{H}$ (Block-Diagonal-Aufbau gem6aB (3.21))
 - 2: $\dot{V}_0 = \mathbf{1} + \mathbf{j}\mathbf{0} \in \mathbb{C}^{(\tau \cdot b\phi) \times 1}$; $n = 0$; $\text{tol} = \infty$
 - 3: **while** $\text{tol} \geq \varepsilon$ **und** $n < K$ **do**
 - 4: $\dot{V}_{(n+1)} = \text{solve}(\dot{M}, \dot{V}_{(n)}^{*\circ(-1)} + \dot{H})$ {Sparse-L6ser}
 - 5: $\text{tol} = \max(\|s_{(n+1)} - s_{(n)}\|_2)$; $n = n + 1$
 - 6: **end while**
 - 7: $\mathbf{V}_n = \text{reshape}(\dot{V}_n, b\phi \times \tau)$
 - 8: **return** $\mathbf{V}_n, n$
-

3.3 Limitation: Keine PV-Knoten

Die Effizienz des Tensor Power Flow basiert auf einer fundamentalen Annahme: *Der Leistungsvektor $\mathbf{s}$ ist f6ur alle τ Lastfl6sse vollst6ndig bekannt und bleibt w6hrend der gesamten Iteration konstant.*

Diese Annahme erm6glicht:

1. Die **Vorausberechnung** der Systemmatrix Z_B (bzw. $\dot{M}$) und des Konstantvektors $\mathbf{w}$.
2. Die **Wiederverwendung** dieser Gr6Ben 6ber alle Iterationen und alle τ Lastfl6sse.
3. Die **Zusammenfassung** aller τ Lastfl6sse in eine einzige Matrix-Matrix-Multiplikation.

Ohne diese Konstanzheit w6re der Tensor-Reshape (3.13) nicht durchf6hrbar, da jeder Lastfluss seine eigene, sich 6ndernde Systemmatrix ben6tigen w6rde.

3.3.1 Warum PV-Knoten diese Annahme verletzen

Bei PV-Knoten ist die Blindleistung Q_k unbekannt (vgl. Abschnitt 2.1.3). Der Leistungsvektor am PV-Knoten k lautet:

$$s_k = P_k^{\text{spec}} + j \underbrace{Q_k}_{\text{unbekannt}} \quad (3.22)$$

Da Q_k iterativ bestimmt werden muss, um die Spannungsbedingung $|V_k| = |V_k^{\text{spec}}|$ zu erfüllen, ändert sich s_k zwischen den Iterationen. Dies hat folgende Konsequenzen für die TPF-Formulierung:

1. Die Matrix $\mathbf{A} = \text{diag}(\alpha_P \odot \mathbf{s}^*)$ aus (3.3) ist **nicht mehr konstant**, da $\mathbf{s}$ sich ändert.
2. Damit sind auch $\mathbf{F} = -\mathbf{B}^{-1}\mathbf{A}$ und $\mathbf{w} = -\mathbf{B}^{-1}\mathbf{c}$ (Letzteres über $\mathbf{c}$) nicht mehr über die Iterationen konstant.
3. Im Fall rein konstanter Leistung ($\alpha_P = 1$): Die Iteration $\mathbf{v}_{(n+1)} = Z_B(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*o(-1)} + \mathbf{w})$ ändert sich nicht in der Systemmatrix Z_B selbst, aber der Leistungsvektor $\dot{S}$ im Produkt $\dot{S}^* \odot \dot{V}_{(n)}^{*o(-1)}$ ist für PV-Knoten iterationsabhängig.
4. Im Tensor-Setting: Verschiedene Szenarien i und j haben im Allgemeinen unterschiedliche Q -Werte am selben PV-Knoten. Die Matrix $\dot{S}$ ändert sich daher sowohl über die Iterationen als auch – nach jeder Q -Korrektur – szenarioabhängig.

Die Limitation des TPF auf PQ-Knoten ist in der Praxis für Verteilnetze oft akzeptabel, da dort die Mehrzahl der Einspeiser (PV-Anlagen mit Wechselrichter, Batteriespeicher) als PQ-Knoten mit festem Leistungsfaktor modelliert wird [1]. Mit zunehmender Integration von Synchrongeneratoren und spannungsregelnden Einheiten in Verteilnetzen gewinnt die korrekte Abbildung von PV-Knoten jedoch an Bedeutung. Die Entwicklung geeigneter Erweiterungen des TPF zur Berücksichtigung von PV-Knoten ist Gegenstand von Kapitel 4.

Kapitel 4

Methoden zur PV-Knoten-Integration im Tensor Power Flow

In den vorherigen Kapiteln wurde gezeigt, dass der Tensor Power Flow durch die Konstanz der Systemmatrix $\mathbf{F}$ (bzw. $\mathbf{B}^{-1}$) einen erheblichen Geschwindigkeitsvorteil gegenüber konventionellen Verfahren bietet. Die zentrale Limitation liegt in der Annahme, dass der Leistungsvektor $\mathbf{s}$ über alle Iterationen vollständig bekannt und konstant ist (?? und Abschnitt 3.3). PV-Knoten verletzen diese Annahme, da ihre Blindleistung Q_k unbekannt ist und iterativ bestimmt werden muss.

Dieses Kapitel entwickelt zwei Methoden, die PV-Knoten in die FPI-Struktur des TPF integrieren, ohne die Parallelisierbarkeit wesentlich einzuschränken. Beide Methoden operieren auf der allgemeinen FPI-Formulierung aus Abschnitt 3.1:

$$\mathbf{v}_{(n+1)} = \mathbf{F}^{(n)} \cdot \mathbf{v}_{(n)}^{*o(-1)} + \mathbf{w}^{(n)} \quad (4.1)$$

mit den in (3.3)–(3.7) definierten Hilfsgrößen. Der Index (n) kennzeichnet, dass diese Größen im allgemeinen Fall von der aktuellen Schätzung des Leistungsvektors $\mathbf{s}^{(n)}$ abhängen, welcher sich bei PV-Knoten zwischen den Iterationen ändert, da dort $s_k^{(n)} = P_k^{\text{spec}} + jQ_k^{(n)}$ mit iterativ bestimmtem $Q_k^{(n)}$ gilt.

4.1 Analyse der Auswirkung von PV-Knoten auf die FPI-Struktur

4.1.1 Welche Matrizen werden von Q-Änderungen beeinflusst?

Um die nachfolgenden Methoden zu motivieren, wird zunächst systematisch analysiert, welche Bestandteile der FPI-Formulierung (??)–(??) von einer Änderung der Blindleistung Q_k an einem PV-Knoten k betroffen sind. Der Leistungsvektor ändert sich am PV-Knoten gemäß:

$$s_k^{(n)} = P_k^{\text{spec}} + jQ_k^{(n)} \quad \Rightarrow \quad s_k^{*(n)} = P_k^{\text{spec}} - jQ_k^{(n)} \quad (4.2)$$

Die Auswirkungen auf die einzelnen Matrizen sind in Tabelle 4.1 zusammengefasst.

Tabelle 4.1: Abhängigkeit der FPI-Bestandteile von der Blindleistung Q_k eines PV-Knotens für den allgemeinen ZIP-Fall.

Größe	Betroffen?	Bedingung für Konstanz	Begründung
Y_{dd}	Nein	Immer konstant	Nur Netztopologie
$\mathbf{A} = \text{diag}(\alpha_P \odot \mathbf{s}^*)$	Ja ⁽¹⁾	$\alpha_{P,k} = 0$	k -tes Element: $\alpha_{P,k} s_k^*$
$\mathbf{B} = \text{diag}(\alpha_Z \odot \mathbf{s}^*) + Y_{dd}$	Ja ⁽²⁾	$\alpha_{Z,k} = 0$	k -tes Diag.: $\alpha_{Z,k} s_k^*$
$\mathbf{c} = Y_{ds} \mathbf{v}_s + \alpha_I \odot \mathbf{s}^*$	Ja ⁽³⁾	$\alpha_{I,k} = 0$	k -tes Element: $\alpha_{I,k} s_k^*$
$\mathbf{B}^{-1}$	Ja ⁽²⁾	$\alpha_{Z,k} = 0$	Folgt aus $\mathbf{B}$
$\mathbf{F} = -\mathbf{B}^{-1} \mathbf{A}$	Ja ^(1,2)	$\alpha_{P,k} = \alpha_{Z,k} = 0$	Folgt aus $\mathbf{A}, \mathbf{B}$
$\mathbf{w} = -\mathbf{B}^{-1} \mathbf{c}$	Ja ^(2,3)	$\alpha_{Z,k} = \alpha_{I,k} = 0$	Folgt aus $\mathbf{B}, \mathbf{c}$

Interpretation. Die Auswirkungen einer Q-Änderung am PV-Knoten hängen fundamental vom Lastmodell an diesem Knoten ab:

- **Fall 1:** $\alpha_{Z,k} = 0$ am PV-Knoten. Dann bleibt $\mathbf{B}$ (und damit $\mathbf{B}^{-1}$) über alle Iterationen *konstant*. Lediglich $\mathbf{A}$ und ggf. $\mathbf{c}$ ändern sich. Dies ist der rechentechnisch günstigste Fall, da die Inversion von $\mathbf{B}$ (die teuerste Operation) nur einmal durchgeführt werden muss.
- **Fall 2:** $\alpha_{Z,k} \neq 0$ am PV-Knoten. Dann ändert sich $\mathbf{B}$ mit jeder Q-Aktualisierung, und $\mathbf{B}^{-1}$ muss grundsätzlich neu berechnet werden. Jedoch ändert sich nur ein *Diagonalelement* von $\mathbf{B}$, was effiziente Rang-1-Updates (Sherman-Morrison-Formel) ermöglicht.

In der Praxis werden Generatoren an PV-Knoten fast ausschließlich als leistungskonstante Einspeiser modelliert ($\alpha_{P,k} = 1, \alpha_{Z,k} = \alpha_{I,k} = 0$), da ihr Verhalten

durch die Leistungselektronik oder Turbinenregelung bestimmt wird, nicht durch die Netzspannung [2, 5]. Für diesen häufigen Spezialfall gilt:

$$\alpha_{Z,k} = \alpha_{I,k} = 0 \quad \Rightarrow \quad \mathbf{B}^{-1} \text{ und } \mathbf{w} \text{ bleiben konstant.} \quad (4.3)$$

Die nachfolgende Entwicklung wird zunächst für den *allgemeinen ZIP-Fall* formuliert und die Vereinfachungen für den Spezialfall (4.3) jeweils explizit angegeben.

4.1.2 Umschreibung der Iteration

Für die nachfolgende Analyse ist es hilfreich, die Iteration (4.1) explizit in Abhängigkeit des Leistungsvektors auszuschreiben. Einsetzen von (??)–(??) ergibt:

$$\mathbf{v}_{(n+1)} = -(\mathbf{B}^{(n)})^{-1} \cdot [\mathbf{A}^{(n)} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{c}^{(n)}] \quad (4.4)$$

Da $\mathbf{A}^{(n)}$ eine Diagonalmatrix ist, kann das Produkt $\mathbf{A}^{(n)} \cdot \mathbf{v}_{(n)}^{*\circ(-1)}$ als Hadamard-Operation umgeschrieben werden:

$$\mathbf{A}^{(n)} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} = (\alpha_P \odot \mathbf{s}^{*(n)}) \odot \mathbf{v}_{(n)}^{*\circ(-1)} \quad (4.5)$$

Damit wird (4.4) zu:

$$\mathbf{v}_{(n+1)} = -(\mathbf{B}^{(n)})^{-1} \cdot [(\alpha_P \odot \mathbf{s}^{*(n)}) \odot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{c}^{(n)}] \quad (4.6)$$

In dieser Form wird deutlich, dass die Änderung von $\mathbf{s}^{(n)}$ an PV-Knoten:

1. den Vektor $(\alpha_P \odot \mathbf{s}^{*(n)}) \odot \mathbf{v}_{(n)}^{*\circ(-1)}$ beeinflusst (über α_P -gewichteten Anteil),
2. den Vektor $\mathbf{c}^{(n)}$ beeinflusst (über α_I -gewichteten Anteil),
3. die Matrix $(\mathbf{B}^{(n)})^{-1}$ beeinflusst (über α_Z -gewichteten Anteil).

4.1.3 Konsequenzen für den Konvergenzbeweis

Der Konvergenzbeweis nach Theorem 2.3 basiert auf der Konstanz von $\mathbf{s}$ und damit aller Hilfsmatrizen. Wenn $\mathbf{s}$ sich zwischen den Iterationen ändert, ist der Kontraktionsfaktor η aus (2.78) nicht mehr über die gesamte Iteration konstant. Es gelten folgende Einschränkungen:

1. Der formale Konvergenzbeweis (Theorem 2.3) ist in seiner vorliegenden Form *nicht direkt anwendbar*, da die Abbildung $T^{(n)}(\cdot)$ iterationsabhängig wird.

2. Die physikalische Konvergenzbedingung $\|Z_{ii}\| < \|Z_{L,i}\|$ (Abschnitt 2.4.5) bleibt als heuristische Richtlinie anwendbar, sofern die Q-Änderungen die äquivalente Lastimpedanz nicht fundamental verändern.
3. In der Praxis konvergieren die nachfolgend vorgestellten Methoden für typische Verteilnetz-Betriebspunkte zuverlässig, wie in Kapitel 6 numerisch validiert wird.

4.2 Methode A: Äußere Q-Schleife

4.2.1 Grundidee

Methode A realisiert die PV-Knoten-Behandlung durch eine verschachtelte Schleifenstruktur. Der Standard-TPF wird als *innere Schleife* mit festem Leistungsvektor $\mathbf{s}^{(\ell)}$ (und damit festen Matrizen $\mathbf{F}^{(\ell)}$, $\mathbf{w}^{(\ell)}$) ausgeführt, bis er konvergiert. Anschließend wird in einer *äußeren Schleife* die Blindleistung Q_k an den PV-Knoten korrigiert, der Leistungsvektor $\mathbf{s}^{(\ell+1)}$ aktualisiert, und die Hilfsmatrizen neu berechnet.

Der konzeptionelle Ablauf ist:

1. Setze $Q_k^{(0)} = 0$ als initialen Schätzwert (äußere Iteration $\ell = 0$).
2. Berechne $\mathbf{s}^{(\ell)}$, daraus $\mathbf{A}^{(\ell)}$, $\mathbf{B}^{(\ell)}$, $\mathbf{c}^{(\ell)}$, $\mathbf{F}^{(\ell)}$, $\mathbf{w}^{(\ell)}$.
3. Führe den vollständigen TPF (innere Schleife) mit festem $\mathbf{F}^{(\ell)}$, $\mathbf{w}^{(\ell)}$ bis zur Konvergenz aus $\rightarrow$ ergibt $\mathbf{v}^{(\ell)}$.
4. Berechne die Abweichung $\Delta|V_k|^2 = |V_k^{\text{spec}}|^2 - |v_k^{(\ell)}|^2$.
5. Korrigiere $Q_k^{(\ell+1)} = Q_k^{(\ell)} + \Delta Q_k^{(\ell)}$.
6. Falls $|\Delta|V_k|^2| < \varepsilon_Q$ für alle PV-Knoten: fertig. Sonst: zurück zu Schritt 2.

Ein wesentlicher Vorteil dieser Struktur ist, dass die innere Schleife ein *unmodifizierter* Standard-TPF ist: Da $\mathbf{s}^{(\ell)}$ dort konstant gehalten wird, sind alle Hilfsmatrizen $\mathbf{A}^{(\ell)}$, $\mathbf{B}^{(\ell)}$, $\mathbf{F}^{(\ell)}$, $\mathbf{w}^{(\ell)}$ über die inneren Iterationen fixiert. Der Konvergenzbeweis (Theorem 2.3) ist daher für die innere Schleife vollständig gültig.

4.2.2 Herleitung der Q-Korrekturformel

Für die Q-Korrektur in Schritt 5 wird eine Beziehung zwischen ΔQ_k und dem Spannungsmismatch $\Delta|V_k|^2$ benötigt. Diese wird über das Thévenin-Ersatzschaltbild am PV-Knoten k hergeleitet.

Thévenin-Äquivalent am Knoten k . Aus Sicht eines PV-Knotens k lässt sich das restliche Netz durch sein Thévenin-Äquivalent beschreiben. Die Spannung am Knoten k ergibt sich als:

$$V_k = E_{\text{th},k} - Z_{B,kk}^{(\ell)} \cdot I_k \quad (4.7)$$

wobei $E_{\text{th},k}$ die Thévenin-Leerlaufspannung (bestimmt durch das restliche Netz und die Slack-Spannung) und $Z_{B,kk}^{(\ell)}$ das k -te Diagonalelement der Bus-Impedanzmatrix ist:

$$Z_{B,kk}^{(\ell)} = \left[(\mathbf{B}^{(\ell)})^{-1} \right]_{kk} = R_{kk}^{(\ell)} + jX_{kk}^{(\ell)} \quad (4.8)$$

Strominjektion als Funktion von Q_k . Die komplexe Leistungsinjektion am Knoten k in Erzeugerkonvention lautet $S_k = P_k + jQ_k = V_k \cdot I_k^*$. Daraus folgt für den Strom:

$$I_k = \frac{S_k^*}{V_k^*} = \frac{P_k - jQ_k}{V_k^*} \quad (4.9)$$

Linearisierung: Stromänderung bei Q-Variation. Bei festgehaltenem P_k und unter der Annahme, dass sich V_k bei einer kleinen Änderung ΔQ_k nur geringfügig ändert (Linearisierung um den aktuellen Arbeitspunkt $v_k^{(\ell)}$), ergibt sich die Stromänderung:

$$\Delta I_k \approx \left. \frac{-j \Delta Q_k}{V_k^*} \right|_{V_k=v_k^{(\ell)}} \quad (4.10)$$

Spannungsänderung über Thévenin-Impedanz. Die resultierende Spannungsänderung am Knoten k folgt aus dem Thévenin-Äquivalent (4.7):

$$\Delta V_k = -Z_{B,kk}^{(\ell)} \cdot \Delta I_k = Z_{B,kk}^{(\ell)} \cdot \frac{j \Delta Q_k}{(v_k^{(\ell)})^*} \quad (4.11)$$

Mit $Z_{B,kk}^{(\ell)} = R_{kk}^{(\ell)} + jX_{kk}^{(\ell)}$ und $(v_k^{(\ell)})^* = |v_k^{(\ell)}| e^{-j\theta_k}$ ergibt die Ausmultiplikation:

$$\Delta V_k = \frac{\Delta Q_k}{|v_k^{(\ell)}|} \left(X_{kk}^{(\ell)} - jR_{kk}^{(\ell)} \right) e^{j\theta_k} \quad (4.12)$$

Änderung des Spannungsbetragsquadrats. Die Änderung von $|V_k|^2$ lässt sich über die Identität

$$\Delta(|V_k|^2) = 2 \operatorname{Re}(V_k^* \cdot \Delta V_k) \quad (4.13)$$

berechnen. Einsetzen von (4.12) mit $V_k^* = |v_k^{(\ell)}| e^{-j\theta_k}$ liefert:

$$\begin{aligned} V_k^* \cdot \Delta V_k &= |v_k^{(\ell)}| e^{-j\theta_k} \cdot \frac{\Delta Q_k}{|v_k^{(\ell)}|} \left(X_{kk}^{(\ell)} - jR_{kk}^{(\ell)} \right) e^{j\theta_k} \\ &= \Delta Q_k \cdot \left(X_{kk}^{(\ell)} - jR_{kk}^{(\ell)} \right) \end{aligned} \quad (4.14)$$

Der Realteil ergibt:

$$\operatorname{Re}(V_k^* \cdot \Delta V_k) = X_{kk}^{(\ell)} \cdot \Delta Q_k \quad (4.15)$$

Bemerkenswert ist, dass der Widerstandsanteil $R_{kk}^{(\ell)}$ ausschließlich zum Imaginärteil beiträgt und sich somit vollständig herauskürzt. Einsetzen in (4.13) liefert die zentrale Sensitivitätsbeziehung:

$$\Delta(|V_k|^2) = 2 X_{kk}^{(\ell)} \cdot \Delta Q_k \quad (4.16)$$

Ergebnis: Q-Korrekturformel. Um die Spannung vom berechneten Wert $|v_k^{(\ell)}|$ auf den Sollwert $|V_k^{\text{spec}}|$ zu korrigieren, wird $\Delta(|V_k|^2) = |V_k^{\text{spec}}|^2 - |v_k^{(\ell)}|^2$ gefordert. Auflösen von (4.16) nach ΔQ_k ergibt:

$$\Delta Q_k^{(\ell)} = \frac{|V_k^{\text{spec}}|^2 - |v_k^{(\ell)}|^2}{2 \cdot X_{kk}^{(\ell)}} \quad (4.17)$$

wobei $X_{kk}^{(\ell)} = \operatorname{Im} \left(\left[(\mathbf{B}^{(\ell)})^{-1} \right]_{kk} \right)$ die Thévenin-Reaktanz am Knoten k ist.

4.2.3 Eigenschaften der Korrekturformel

Die hergeleitete Formel (4.17) besitzt mehrere bemerkenswerte Eigenschaften:

1. **Spannungsunabhängigkeit:** Die Formel hängt nur von $X_{kk}^{(\ell)}$ und den Spannungsbeträgen ab, nicht vom aktuellen Spannungswinkel θ_k . Dies ist eine Konsequenz der vollständigen Eliminierung des Widerstandsanteils R_{kk} in Gleichung (4.15).
2. **Physikalische Interpretation:** Die Sensitivität $\partial|V_k|^2/\partial Q_k = 2X_{kk}$ besagt, dass die Spannungsänderung proportional zur Thévenin-Reaktanz ist. In Netzen mit hoher Reaktanz (typisch für Übertragungsnetze) reicht bereits eine geringe Q-Änderung für eine signifikante Spannungs Korrektur.
3. **Abhängigkeit von $\mathbf{B}^{(\ell)}$:**
 - Wenn $\alpha_{Z,k} = 0$ (am PV-Knoten): $\mathbf{B}^{(\ell)} = \mathbf{B}$ ist konstant, und damit auch $X_{kk}^{(\ell)} = X_{kk}$ über alle äußeren Iterationen. Die Sensitivität ist eine *vorab berechenbare Konstante*.
 - Wenn $\alpha_{Z,k} \neq 0$: $\mathbf{B}^{(\ell)}$ ändert sich mit $Q_k^{(\ell)}$, und $X_{kk}^{(\ell)}$ muss nach jeder äußeren Iteration aktualisiert werden. In der Praxis ist die Änderung gering und $X_{kk}^{(\ell)} \approx X_{kk}^{(0)}$ eine gute Näherung.
4. **Lineare Konvergenz:** Da (4.17) eine First-Order-Sensitivität darstellt (Linearisierung um den aktuellen Arbeitspunkt), konvergiert die äußere Schleife

linear. Dies ist konsistent mit dem beobachteten Verhalten in den numerischen Tests (??).

4.2.4 Algorithmus

Algorithm 4 Methode A: Äußere Q-Schleife mit innerem TPF (allgemeiner ZIP-Fall)

Require: $\mathbf{s}_{\text{PQ}}$, P_k^{spec} und $|V_k^{\text{spec}}|$ für $k \in \Omega_{\text{PV}}$, Y_{dd} , Y_{ds} , $\mathbf{v}_s$, ZIP-Koeffizienten $\alpha_P, \alpha_I, \alpha_Z$, Toleranzen $\varepsilon_v, \varepsilon_Q$, max. äußere Iterationen L

Ensure: $\mathbf{v}$, Q_k für $k \in \Omega_{\text{PV}}$

```

1: Initialisierung:
2:  $Q_k^{(0)} = 0$  für alle  $k \in \Omega_{\text{PV}}$ ;  $\ell = 0$ 
3:
4: Äußere Schleife:
5: while  $\max_k |\Delta|V_k|^2| \geq \varepsilon_Q$  und  $\ell < L$  do
6:   % Leistungsvektor und Hilfsmatrizen aktualisieren
7:    $\mathbf{s}_k^{(\ell)} = P_k^{\text{spec}} + \mathbf{j}Q_k^{(\ell)}$  für alle  $k \in \Omega_{\text{PV}}$ 
8:    $\mathbf{A}^{(\ell)} = \text{diag}(\alpha_P \odot \mathbf{s}^{*(\ell)})$ 
9:    $\mathbf{B}^{(\ell)} = \text{diag}(\alpha_Z \odot \mathbf{s}^{*(\ell)}) + Y_{dd}$ 
10:   $\mathbf{c}^{(\ell)} = Y_{ds}\mathbf{v}_s + \alpha_I \odot \mathbf{s}^{*(\ell)}$ 
11:  Berechne  $(\mathbf{B}^{(\ell)})^{-1}$  {Nur wenn  $\alpha_{Z,k} \neq 0$ ; sonst einmalig}
12:   $\mathbf{F}^{(\ell)} = -(\mathbf{B}^{(\ell)})^{-1}\mathbf{A}^{(\ell)}$ ;  $\mathbf{w}^{(\ell)} = -(\mathbf{B}^{(\ell)})^{-1}\mathbf{c}^{(\ell)}$ 
13:   $X_{kk}^{(\ell)} = \text{Im} \left( [(\mathbf{B}^{(\ell)})^{-1}]_{kk} \right)$  für alle  $k \in \Omega_{\text{PV}}$ 
14:
15:  % Innere Schleife: Standard-FPI mit konstantem  $\mathbf{F}^{(\ell)}$ ,  $\mathbf{w}^{(\ell)}$ 
16:   $\mathbf{v}^{(0)} = \mathbf{1} + \mathbf{j}\mathbf{0}$ ;  $n = 0$ ;  $\text{tol}_v = \infty$ 
17:  while  $\text{tol}_v \geq \varepsilon_v$  und  $n < K$  do
18:     $\mathbf{v}^{(n+1)} = \mathbf{F}^{(\ell)} \cdot \mathbf{v}_{(n)}^{*o(-1)} + \mathbf{w}^{(\ell)}$ 
19:     $\text{tol}_v = \max \|\mathbf{s}^{(\ell)} - \mathbf{s}_{\text{calc}}^{(n+1)}\|$ ;  $n = n + 1$ 
20:  end while
21:   $\mathbf{v}^{(\ell)} = \mathbf{v}^{(n)}$  {Konvergierte Lösung der inneren Schleife}
22:
23:  % Q-Korrektur
24:  for jeden PV-Knoten  $k \in \Omega_{\text{PV}}$  do
25:     $\Delta|V_k|^2 = |V_k^{\text{spec}}|^2 - |v_k^{(\ell)}|^2$ 
26:     $\Delta Q_k^{(\ell)} = \Delta|V_k|^2 / (2 \cdot X_{kk}^{(\ell)})$ 
27:     $Q_k^{(\ell+1)} = Q_k^{(\ell)} + \Delta Q_k^{(\ell)}$ 
28:     $Q_k^{(\ell+1)} = \max \left( Q_k^{\min}, \min(Q_k^{\max}, Q_k^{(\ell+1)}) \right)$  {Q-Limits}
29:  end for
30:   $\ell = \ell + 1$ 
31: end while
32: return  $\mathbf{v}^{(\ell)}$ ,  $\{Q_k^{(\ell)}\}_{k \in \Omega_{\text{PV}}}$ 

```

4.2.5 Rechentechnische Betrachtung

Fall $\alpha_{Z,k} = 0$ am PV-Knoten (häufiger Spezialfall). Die Matrix $\mathbf{B}$ ist konstant und $\mathbf{B}^{-1}$ muss nur einmal berechnet werden. In der äußeren Schleife müssen lediglich die Diagonalmatrix $\mathbf{A}^{(\ell)}$, der Vektor $\mathbf{c}^{(\ell)}$ und die Produkte $\mathbf{F}^{(\ell)} = -\mathbf{B}^{-1}\mathbf{A}^{(\ell)}$, $\mathbf{w}^{(\ell)} = -\mathbf{B}^{-1}\mathbf{c}^{(\ell)}$ neu berechnet werden. Die Berechnung von $\mathbf{F}^{(\ell)}$ ist eine Matrix-Diagonalmatrix-Multiplikation (Skalierung der Spalten von $\mathbf{B}^{-1}$) mit Kosten $\mathcal{O}((b\phi)^2)$.

Fall $\alpha_{Z,k} \neq 0$ (allgemeiner Fall). Die Matrix $\mathbf{B}$ ändert sich, da ihr Diagonalelement am PV-Knoten k den Wert $\alpha_{Z,k}s_k^{*(\ell)} + Y_{dd,kk}$ hat. Eine vollständige Neuberechnung von $\mathbf{B}^{-1}$ hat Kosten $\mathcal{O}((b\phi)^3)$. Stattdessen kann die Sherman-Morrison-Formel für Rang-1-Updates verwendet werden:

$$(\mathbf{B}^{(\ell+1)})^{-1} = (\mathbf{B}^{(\ell)} + \delta_k \mathbf{e}_k \mathbf{e}_k^T)^{-1} = (\mathbf{B}^{(\ell)})^{-1} - \frac{(\mathbf{B}^{(\ell)})^{-1} \mathbf{e}_k \mathbf{e}_k^T (\mathbf{B}^{(\ell)})^{-1} \cdot \delta_k}{1 + \delta_k \mathbf{e}_k^T (\mathbf{B}^{(\ell)})^{-1} \mathbf{e}_k} \quad (4.18)$$

mit $\delta_k = \alpha_{Z,k}(s_k^{*(\ell+1)} - s_k^{*(\ell)}) = -j \alpha_{Z,k} \Delta Q_k^{(\ell)}$ und $\mathbf{e}_k$ dem k -ten Einheitsvektor. Die Kosten pro PV-Knoten reduzieren sich damit auf $\mathcal{O}((b\phi)^2)$.

Konvergenz der inneren Schleife. Da innerhalb der inneren Schleife $\mathbf{s}^{(\ell)}$ und damit $\mathbf{F}^{(\ell)}$ konstant sind, ist der Banach-Fixpunktsatz (Theorem 2.3) anwendbar. Die innere Schleife konvergiert garantiert, solange $\eta^{(\ell)} < 1$ mit:

$$\eta^{(\ell)} = \left\| (\mathbf{B}^{(\ell)})^{-1} \text{diag}(\alpha_P \odot \mathbf{s}^{*(\ell)}) \right\|_1 \cdot \frac{1}{v_{\min}^2} \quad (4.19)$$

4.3 Methode B: Eingebettete Q-Korrektur

4.3.1 Grundidee

Im Gegensatz zu Methode A, die eine verschachtelte Schleifenstruktur erfordert, integriert Methode B die PV-Knoten-Behandlung direkt in *jede* FPI-Iteration. Die Grundidee besteht aus drei Schritten, die nach dem regulären FPI-Update ausgeführt werden:

1. **Spannungsbetrags-Projektion:** Der an einem PV-Knoten berechnete Spannungszeiger wird in seiner Richtung (Winkel) beibehalten, aber im Betrag auf den Sollwert $|V_k^{\text{spec}}|$ skaliert.
2. **Q-Rückrechnung:** Die Blindleistung wird aus der Netzgleichung (Kirchhoff'sches Knotengesetz) mit dem projizierten Spannungszeiger exakt zurückberechnet.

3. **Hilfsmatrizen-Update:** Die Matrizen $\mathbf{A}$, $\mathbf{B}$ (falls nötig), $\mathbf{c}$, $\mathbf{F}$, $\mathbf{w}$ werden mit dem neuen Q_k aktualisiert.

Dieser Ansatz erzwingt die Spannungsbedingung $|v_k| = |V_k^{\text{spec}}|$ in jeder Iteration *exakt*. Die Blindleistung ergibt sich direkt aus der Leistungsbilanz, ohne Linearisierung oder Sensitivitätsapproximation.

4.3.2 Mathematische Formulierung

Schritt 1: FPI-Schritt mit aktuellen Hilfsmatrizen. Die reguläre FPI-Iteration (4.1) wird mit den aktuellen Matrizen $\mathbf{F}^{(n)}$, $\mathbf{w}^{(n)}$ durchgeführt:

$$\mathbf{v}'_{(n+1)} = \mathbf{F}^{(n)} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{w}^{(n)} \quad (4.20)$$

Der Strich in $\mathbf{v}'_{(n+1)}$ kennzeichnet, dass dies ein Zwischenergebnis ist, das an den PV-Knoten noch korrigiert wird. An PQ-Knoten gilt $v_{k,(n+1)} = v'_{k,(n+1)}$ ohne Korrektur.

Schritt 2: Spannungsbetrags-Projektion. An jedem PV-Knoten $k \in \Omega_{\text{PV}}$ wird der Spannungszeiger auf den Sollbetrag projiziert:

$$v_{k,(n+1)} = |V_k^{\text{spec}}| \cdot \frac{v'_{k,(n+1)}}{|v'_{k,(n+1)}|} \quad (4.21)$$

Dies ist eine reine Skalierungsoperation. Der resultierende Spannungszeiger hat den korrekten Betrag $|v_{k,(n+1)}| = |V_k^{\text{spec}}|$ und einen aus der FPI-Iteration stammenden Winkel.

Schritt 3: Q-Rückrechnung aus dem Kirchhoff'schen Knotengesetz. Nach der Projektion ist die Spannung am PV-Knoten bekannt. Der zugehörige Strom ergibt sich aus der Admittanzmatrix:

$$i_{k,(n+1)} = \sum_{m \in \Omega_d} Y_{dd,km} \cdot v_{m,(n+1)} + (Y_{ds})_k \cdot \mathbf{v}_s \quad (4.22)$$

Die komplexe Leistung am PV-Knoten ergibt sich zu:

$$s_{k,(n+1)}^{\text{calc}} = v_{k,(n+1)} \cdot i_{k,(n+1)}^* \quad (4.23)$$

Die Blindleistung ist der Imaginärteil:

$$Q_k^{(n+1)} = \text{Im} \left(v_{k,(n+1)} \cdot i_{k,(n+1)}^* \right) \quad (4.24)$$

Es sei angemerkt, dass die so berechnete Blindleistung die Netzgleichung *exakt* erfüllt – im Gegensatz zur linearisierten Sensitivitätsformel (4.17) aus Methode A.

Schritt 4: Leistungsvektor und Hilfsmatrizen aktualisieren. Der Leistungsvektor wird aktualisiert:

$$\mathbf{s}_k^{(n+1)} = P_k^{\text{spec}} + \mathrm{j}Q_k^{(n+1)}, \quad \forall k \in \Omega_{\text{PV}} \quad (4.25)$$

Anschließend werden die Hilfsmatrizen für die nächste Iteration neu berechnet:

$$\mathbf{A}^{(n+1)} = \text{diag}(\alpha_P \odot \mathbf{s}^{*(n+1)}) \quad (4.26)$$

$$\mathbf{B}^{(n+1)} = \text{diag}(\alpha_Z \odot \mathbf{s}^{*(n+1)}) + Y_{dd} \quad (4.27)$$

$$\mathbf{c}^{(n+1)} = Y_{ds}\mathbf{v}_s + \alpha_I \odot \mathbf{s}^{*(n+1)} \quad (4.28)$$

$$\mathbf{F}^{(n+1)} = -(\mathbf{B}^{(n+1)})^{-1}\mathbf{A}^{(n+1)} \quad (4.29)$$

$$\mathbf{w}^{(n+1)} = -(\mathbf{B}^{(n+1)})^{-1}\mathbf{c}^{(n+1)} \quad (4.30)$$

Vereinfachung für $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten. In diesem häufigen Spezialfall sind $\mathbf{B}$, $\mathbf{B}^{-1}$ und $\mathbf{c}$ konstant. Es müssen lediglich die Einträge von $\mathbf{A}$ an den PV-Knoten-Positionen aktualisiert werden, und $\mathbf{F}$ ergibt sich durch Neuskalierung der entsprechenden Spalten von $\mathbf{B}^{-1}$. Die Berechnung von $\mathbf{w}$ entfällt, da $\mathbf{c}$ und $\mathbf{B}^{-1}$ unverändert bleiben.

4.3.3 Algorithmus

Algorithm 5 Methode B: Eingebettete Q-Korrektur (allgemeiner ZIP-Fall)

Require: $\mathbf{s}_{\text{PQ}}$, P_k^{spec} und $|V_k^{\text{spec}}|$ für $k \in \Omega_{\text{PV}}$, Y_{dd} , Y_{ds} , $\mathbf{v}_s$, ZIP-Koeffizienten, Toleranz ε , max. Iterationen K

Ensure: $\mathbf{v}$, Q_k für $k \in \Omega_{\text{PV}}$

```

1: Initialisierung:
2:  $Q_k^{(0)} = 0$  für alle  $k \in \Omega_{\text{PV}}$ 
3:  $\mathbf{v}^{(0)} = \mathbf{1} + \mathbf{j}0$ ;  $n = 0$ ;  $\text{tol} = \infty$ 
4:  $\mathbf{s}^{(0)} = [\mathbf{s}_{\text{PQ}}; P_k^{\text{spec}} + \mathbf{j}Q_k^{(0)}]$ 
5: Berechne  $\mathbf{A}^{(0)}$ ,  $\mathbf{B}^{(0)}$ ,  $\mathbf{c}^{(0)}$ ,  $(\mathbf{B}^{(0)})^{-1}$ ,  $\mathbf{F}^{(0)}$ ,  $\mathbf{w}^{(0)}$  gemäß (??)–(??)
6:
7: Hauptschleife:
8: while  $\text{tol} \geq \varepsilon$  und  $n < K$  do
9:   % Schritt 1: FPI-Schritt
10:   $\mathbf{v}'_{(n+1)} = \mathbf{F}^{(n)} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{w}^{(n)}$ 
11:
12:  % Schritt 2: Spannungsbetrags-Projektion an PV-Knoten
13:  for jeden PV-Knoten  $k \in \Omega_{\text{PV}}$  do
14:     $v_{k,(n+1)} = |V_k^{\text{spec}}| \cdot v'_{k,(n+1)} / |v'_{k,(n+1)}|$ 
15:  end for
16:   $v_{m,(n+1)} = v'_{m,(n+1)}$  für alle  $m \notin \Omega_{\text{PV}}$ 
17:
18:  % Schritt 3: Q-Rückrechnung aus Netzgleichung
19:  for jeden PV-Knoten  $k \in \Omega_{\text{PV}}$  do
20:     $i_{k,(n+1)} = \sum_m Y_{dd,km} \cdot v_{m,(n+1)} + (Y_{ds})_k \cdot \mathbf{v}_s$ 
21:     $Q_k^{(n+1)} = \text{Im} \left( v_{k,(n+1)} \cdot i_{k,(n+1)}^* \right)$ 
22:     $Q_k^{(n+1)} = \max \left( Q_k^{\min}, \min(Q_k^{\max}, Q_k^{(n+1)}) \right)$  {Q-Limits}
23:  end for
24:
25:  % Schritt 4: Leistungsvektor und Hilfsmatrizen aktualisieren
26:   $\mathbf{s}_k^{(n+1)} = P_k^{\text{spec}} + \mathbf{j}Q_k^{(n+1)}$  für alle  $k \in \Omega_{\text{PV}}$ 
27:  Aktualisiere  $\mathbf{A}^{(n+1)}$  gemäß (4.26)
28:  if  $\alpha_{Z,k} \neq 0$  für ein  $k \in \Omega_{\text{PV}}$  then
29:    Aktualisiere  $\mathbf{B}^{(n+1)}$  und  $(\mathbf{B}^{(n+1)})^{-1}$  {Rang-1-Update}
30:  end if
31:  if  $\alpha_{I,k} \neq 0$  für ein  $k \in \Omega_{\text{PV}}$  then
32:    Aktualisiere  $\mathbf{c}^{(n+1)}$ 
33:  end if
34:   $\mathbf{F}^{(n+1)} = -(\mathbf{B}^{(n+1)})^{-1} \mathbf{A}^{(n+1)}$ 
35:   $\mathbf{w}^{(n+1)} = -(\mathbf{B}^{(n+1)})^{-1} \mathbf{c}^{(n+1)}$ 
36:
37:  % Konvergenz prüfen
38:  Berechne Leistungsmismatch an PQ-Knoten
39:   $\text{tol} = \max \|\mathbf{s}_{\text{PQ}}^{\text{spec}} - \mathbf{s}_{\text{PQ,calc}}^{(n+1)}\|$ 
40:   $n = n + 1$ 
41: end while
42: return  $\mathbf{v}_{(n)}$ ,  $\{Q_k^{(n)}\}_{k \in \Omega_{\text{PV}}}$ 

```

4.3.4 Eigenschaften und Diskussion

Vergleich mit Methode A.

- Methode B hat nur eine *einzig*e Schleife. Dies vereinfacht die Implementierung und vermeidet die vollständige Konvergenz einer inneren Schleife bei jedem äußeren Schritt.
- Die Spannungsbedingung $|v_k| = |V_k^{\text{spec}}|$ wird in *jeder* Iteration exakt eingehalten.
- Die Q-Bestimmung (4.24) ist *exakt* (keine Linearisierung), während Methode A die Näherungsformel (4.17) verwendet.
- Die Hilfsmatrizen müssen in jeder Iteration aktualisiert werden (im allgemeinen ZIP-Fall), während sie bei Methode A innerhalb der inneren Schleife konstant bleiben.

Kosten der Matrix-Updates. Die Aktualisierung der Hilfsmatrizen in Schritt 4 hat unterschiedliche Kosten je nach ZIP-Modell am PV-Knoten:

Tabelle 4.2: Kosten der Hilfsmatrizen-Aktualisierung in Methode B pro Iteration.

Operation	$\alpha_{Z,k} = 0$	$\alpha_{Z,k} \neq 0$
$\mathbf{A}^{(n+1)}$: Diagonalelemente setzen	$\mathcal{O}(n_{\text{PV}})$	$\mathcal{O}(n_{\text{PV}})$
$(\mathbf{B}^{(n+1)})^{-1}$: Rang-1-Update	entfällt	$\mathcal{O}((b\phi)^2 \cdot n_{\text{PV}})$
$\mathbf{c}^{(n+1)}$: Element-Update	entfällt*	$\mathcal{O}(n_{\text{PV}})$
$\mathbf{F}^{(n+1)} = -\mathbf{B}^{-1} \mathbf{A}^{(n+1)}$	$\mathcal{O}(b\phi \cdot n_{\text{PV}})^{**}$	$\mathcal{O}((b\phi)^2)$
$\mathbf{w}^{(n+1)} = -\mathbf{B}^{-1} \mathbf{c}^{(n+1)}$	entfällt*	$\mathcal{O}((b\phi)^2)$
Gesamt	$\mathcal{O}(b\phi \cdot n_{\text{PV}})$	$\mathcal{O}((b\phi)^2 \cdot n_{\text{PV}})$

* Falls zusätzlich $\alpha_{I,k} = 0$. ** Da nur n_{PV} Spalten von $\mathbf{F}$ sich ändern (Spalten-Skalierung).

Für den häufigen Spezialfall $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten dominiert die Berechnung von $\mathbf{F}^{(n+1)}$, die sich auf eine Skalierung von n_{PV} Spalten der konstanten Matrix $\mathbf{B}^{-1}$ reduziert. Da $\mathbf{A}$ eine Diagonalmatrix ist, gilt:

$$[\mathbf{F}^{(n+1)}]_{:,k} = -\mathbf{B}_{:,k}^{-1} \cdot A_{kk}^{(n+1)} = -\mathbf{B}_{:,k}^{-1} \cdot \alpha_{P,k} \cdot s_k^{*(n+1)} \quad (4.31)$$

Nur die n_{PV} Spalten von $\mathbf{F}$, die PV-Knoten entsprechen, müssen aktualisiert werden. Die übrigen $(b\phi - n_{\text{PV}})$ Spalten bleiben unverändert.

4.4 Tensorformulierung der Methoden

In diesem Abschnitt wird gezeigt, wie die in Abschnitte 4.2 und 4.3 entwickelten Methoden für die gleichzeitige Berechnung von τ Lastflüssen im Tensor-Rahmen formuliert werden können.

4.4.1 Recap: Tensor-Reshape und allgemeine FPI

Gemäß Abschnitt 3.2 werden alle τ Leistungsvektoren als Spalten der Matrix $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$ angeordnet. Die tensorisierte FPI in ihrer allgemeinen Form lautet:

$$\dot{V}_{(n+1)} = \mathbf{F}^{(n)} \cdot \dot{V}_{(n)}^{*\circ(-1)} + \dot{W}^{(n)} \quad (4.32)$$

wobei $\dot{W}^{(n)} = [\mathbf{w}^{(n)}, \dots, \mathbf{w}^{(n)}] \in \mathbb{C}^{b\phi \times \tau}$ und $\dot{V}_{(n)}^{*\circ(-1)}$ die element-weise Anwendung der Operation “konjugieren und Kehrwert bilden” auf alle Elemente der Matrix darstellt. Im reinen PQ-Fall sind $\mathbf{F}$ und $\mathbf{w}$ konstant über alle Iterationen und alle τ Szenarien. Die Situation ändert sich bei PV-Knoten:

Szenarioabhängigkeit. Im Allgemeinen haben verschiedene Szenarien i und j unterschiedliche Lastbedingungen und damit unterschiedliche Q-Werte am selben PV-Knoten:

$$Q_k^{(n)}[i] \neq Q_k^{(n)}[j] \quad \Rightarrow \quad s_k^{*(n)}[i] \neq s_k^{*(n)}[j] \quad (4.33)$$

Dies bedeutet, dass der Leistungsvektor für verschiedene Szenarien unterschiedlich ist. Folglich wären im allgemeinen Fall die Matrizen $\mathbf{A}$, $\mathbf{B}$, $\mathbf{F}$, $\mathbf{w}$ für *jedes Szenario* unterschiedlich – was die Tensor-Parallelisierung zunichte machen würde.

Die rettende Eigenschaft: Nur $\mathbf{A}$ ist szenarioabhängig (für $\alpha_{Z,k} = 0$). Wenn am PV-Knoten $\alpha_{Z,k} = 0$ gilt, dann hängt $\mathbf{B}$ nicht von $\mathbf{s}$ ab und ist daher für alle Szenarien und Iterationen *identisch*. Ebenso ist $\mathbf{c}$ konstant falls $\alpha_{I,k} = 0$. Der Tensor-Trick bleibt erhalten, indem die szenarioabhängige Matrix $\mathbf{A}^{(n)}$ in die Hadamard-Struktur aufgelöst wird.

4.4.2 Umformulierung für den Tensor-Fall

Für den Spezialfall $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten schreibt sich die Iteration (4.6) als:

$$\mathbf{v}_{(n+1)} = -\mathbf{B}^{-1} \cdot [(\alpha_P \odot \mathbf{s}^{*(n)}) \odot \mathbf{v}_{(n)}^{*\circ(-1)}] - \mathbf{B}^{-1} \mathbf{c} \quad (4.34)$$

wobei $\mathbf{B}^{-1}$ und $\mathbf{c}$ konstant sind. Im Tensor-Format:

$$\dot{V}_{(n+1)} = -\mathbf{B}^{-1} \cdot \underbrace{\left[(\alpha_P \odot \dot{S}^{*(n)}) \odot \dot{V}_{(n)}^{*o(-1)} \right]}_{\in \mathbb{C}^{b\phi \times \tau}} + \dot{W} \quad (4.35)$$

Die Matrix $\dot{S}^{*(n)} \in \mathbb{C}^{b\phi \times \tau}$ enthält die szenario- und iterationsabhängigen Leistungswerte. Entscheidend ist: Die *linke* Matrix $\mathbf{B}^{-1}$ ist für alle τ Szenarien und alle Iterationen *identisch*. Die Multiplikation $\mathbf{B}^{-1} \cdot (\dots)$ bleibt eine einzige Matrix-Matrix-Multiplikation – unabhängig davon, ob sich $\dot{S}^{(n)}$ zwischen den Iterationen ändert.

Allgemeiner ZIP-Fall ($\alpha_{Z,k} \neq 0$). Wenn $\mathbf{B}$ szenarioabhängig wird, ist die Tensor-Parallelisierung in der dense-Formulierung nicht mehr direkt möglich, da jedes Szenario seine eigene Matrix $(\mathbf{B}^{(n)}[i])^{-1}$ benötigen würde. In diesem Fall muss auf die sparse-Formulierung zurückgegriffen werden, bei der die Block-Diagonalmatrix $\dot{M}$ (vgl. (3.21)) ohnehin szenarioabhängige Blöcke enthält. Die einmalige Faktorisierung geht dann zwar verloren, aber die Block-Diagonalstruktur ermöglicht weiterhin eine effiziente parallele Lösung.

In der Praxis ist dieser Fall selten, da Generatoren als leistungskonstant modelliert werden. Für den Rest dieses Abschnitts wird daher $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten vorausgesetzt.

4.4.3 Tensorformulierung von Methode A

Bei Methode A werden alle τ Lastflüsse *gleichzeitig* in der inneren Schleife berechnet. Die Q-Korrektur in der äußeren Schleife wird element-weise über alle τ Szenarien ausgeführt:

Innere Schleife (Matrix-Matrix-Multiplikation). In jeder äußeren Iteration ℓ wird $\mathbf{F}^{(\ell)}$ aktualisiert und die innere Schleife parallelisiert:

$$\dot{V}_{(n+1)} = \mathbf{F}^{(\ell)} \cdot \dot{V}_{(n)}^{*o(-1)} + \dot{W} \quad (4.36)$$

Dies ist eine einzige Multiplikation $\mathbf{F}^{(\ell)} \in \mathbb{C}^{b\phi \times b\phi}$ mit $\dot{V}_{(n)}^{*o(-1)} \in \mathbb{C}^{b\phi \times \tau}$ für alle τ Lastflüsse gleichzeitig. Da $\mathbf{F}^{(\ell)}$ innerhalb der inneren Schleife konstant ist, ist dies exakt die TPF-Operation aus Algorithmus 6.

Bemerkung zur Szenarioabhängigkeit. Im Tensor-Fall hat jedes Szenario i einen anderen Leistungsvektor $\dot{S}[:, i]$ und damit nach der äußeren Q-Korrektur einen anderen Q-Wert $Q_k^{(\ell)}[i]$. Die aktualisierte Matrix $\mathbf{A}^{(\ell)}$ müsste eigentlich szenario-

abhängig sein. Für Methode A im Tensor-Setting wird daher die innere Schleife umformuliert als:

$$\dot{V}_{(n+1)} = -\mathbf{B}^{-1} \cdot [(\alpha_P \odot \dot{S}^{*(\ell)}) \odot \dot{V}_{(n)}^{*o(-1)}] + \dot{W} \quad (4.37)$$

wobei $\dot{S}^{*(\ell)} \in \mathbb{C}^{b\phi \times \tau}$ für jedes Szenario den aktuellen Leistungsvektor enthält und innerhalb der inneren Schleife *fest* ist. Die Operation $(\alpha_P \odot \dot{S}^{*(\ell)}) \odot \dot{V}_{(n)}^{*o(-1)}$ ist eine rein element-weise Operation auf der $b\phi \times \tau$ -Matrix und die anschließende Multiplikation mit $\mathbf{B}^{-1}$ ist eine Matrix-Matrix-Multiplikation. Der Tensor-Vorteil bleibt vollständig erhalten.

Äußere Schleife (element-weise, trivial parallel).

$$\Delta Q[k, :] = \frac{|V_k^{\text{spec}}|^2 - |\dot{V}_{\text{konv}}^{(\ell)}[k, :]|^2}{2 \cdot X_{kk}}, \quad \forall k \in \Omega_{\text{PV}} \quad (4.38)$$

Die Division durch den Skalar $2X_{kk}$ wirkt gleichzeitig auf alle τ Szenarien und ist trivial parallelisierbar.

4.4.4 Tensorformulierung von Methode B

Methode B erfordert keine verschachtelte Schleifenstruktur und ist besonders gut für die Tensorisierung geeignet. Die vier Schritte (Algorithmus 5) werden direkt auf den Tensor-Fall übertragen:

Schritt 1: FPI (Matrix-Matrix-Multiplikation).

$$\dot{V}'_{(n+1)} = -\mathbf{B}^{-1} \cdot [(\alpha_P \odot \dot{S}^{*(n)}) \odot \dot{V}_{(n)}^{*o(-1)}] + \dot{W} \quad (4.39)$$

Die Matrix $\mathbf{B}^{-1}$ bleibt über alle Iterationen und Szenarien konstant (für $\alpha_{Z,k} = 0$). Die szenario- und iterationsabhängige Information steckt vollständig in $\dot{S}^{*(n)}$, welches sich an den PV-Knoten-Zeilen zwischen den Iterationen ändert.

Schritt 2: Projektion (element-weise).

$$\dot{V}_{(n+1)}[k, :] = |V_k^{\text{spec}}| \cdot \frac{\dot{V}'_{(n+1)}[k, :]}{|\dot{V}'_{(n+1)}[k, :]|}, \quad \forall k \in \Omega_{\text{PV}} \quad (4.40)$$

Betragsberechnung und Division wirken element-weise auf die Zeile $[k, :]$ und damit gleichzeitig auf alle τ Szenarien.

Schritt 3: Stromberechnung und Q-Rückrechnung.

$$\dot{I}_{PV} = Y_{dd}[\Omega_{PV}, :] \cdot \dot{V}_{(n+1)} + Y_{ds}[\Omega_{PV}, :] \cdot \mathbf{v}_s \quad (4.41)$$

Dies ist eine Matrix-Matrix-Multiplikation der Dimension $(n_{PV} \times b\phi) \cdot (b\phi \times \tau) = (n_{PV} \times \tau)$. Die Blindleistung ergibt sich element-weise:

$$\dot{Q}_{PV}^{(n+1)} = \text{Im} \left(\dot{V}_{(n+1)}[\Omega_{PV}, :] \odot \dot{I}_{PV}^* \right) \in \mathbb{R}^{n_{PV} \times \tau} \quad (4.42)$$

Schritt 4: $\dot{S}$ -Update (element-weise).

$$\dot{S}^{(n+1)}[k, :] = P_k^{\text{spec}} + j\dot{Q}_{PV}^{(n+1)}[k, :], \quad \forall k \in \Omega_{PV} \quad (4.43)$$

4.4.5 Gesamtvergleich der Tensor-Operationen

Tabelle 4.3: Rechentechnischer Vergleich der Tensor-Operationen pro Iteration (Spezialfall $\alpha_{Z,k} = \alpha_{I,k} = 0$ am PV-Knoten).

Operation	Dimension	Kosten	Gleich wie PQ?
$\mathbf{B}^{-1} \cdot (\dots)$	$(b\phi)^2 \cdot \tau$	Dominant	Ja
Projektion	$n_{PV} \cdot \tau$ (element-weise)	Minimal	–
$Y_{dd,PV} \cdot \dot{V}$	$n_{PV} \cdot b\phi \cdot \tau$	Gering*	–
Q-Berechnung	$n_{PV} \cdot \tau$ (element-weise)	Minimal	–
$\dot{S}$ -Update	$n_{PV} \cdot \tau$ (element-weise)	Minimal	–
$\mathbf{B}^{-1}$ konstant?		Ja	
Tensor-Parallelisierung?		Ja	

* $n_{PV} \ll b\phi$ in typischen Verteilnetzen, daher vernachlässigbar gegenüber der Hauptoperation.

4.4.6 Zusammenfassung: Warum die Parallelisierung erhalten bleibt

Die Analyse zeigt, dass auch bei Verwendung der allgemeinen FPI-Formulierung (4.1) der Tensor-Parallelisierungsvorteil erhalten bleibt, sofern die folgende Bedingung erfüllt ist:

$$\alpha_{Z,k} = 0, \quad \forall k \in \Omega_{PV} \quad (4.44)$$

Unter dieser Bedingung – die für leistungskonstante Generatoren stets erfüllt ist – gilt:

1. Die Matrix $\mathbf{B}^{-1}$ ist **konstant über alle Iterationen und alle τ Szenarien**.

Ihre einmalige Berechnung (oder Faktorisierung im sparse-Fall) wird über die gesamte Rechnung wiederverwendet.

2. Die teure Operation (Matrix-Matrix-Multiplikation $\mathbf{B}^{-1} \cdot (\dots)$) ist **strukturell identisch** zum reinen PQ-Fall und profitiert uneingeschränkt von BLAS/GPU-Parallelisierung.
3. Die Änderung des Leistungsvektors $\dot{S}^{(n)}$ an PV-Knoten betrifft nur die **rechte Seite** des Produkts (den Vektor, der mit $\mathbf{B}^{-1}$ multipliziert wird) und nicht die Matrix selbst.
4. Die zusätzlichen Operationen (Projektion, Stromberechnung, Q-Update) sind entweder element-weise oder von deutlich geringerer Dimension ($n_{PV} \ll b\phi$) und damit vernachlässigbar.
5. Die einzige Einschränkung ist eine erhöhte Iterationszahl: Methode B benötigt typischerweise 8–15 statt 5–10 Iterationen, Methode A benötigt 15–50 FPI-Evaluierungen. Der Speedup pro Iteration bleibt jedoch unverändert.

Wenn $\alpha_{Z,k} \neq 0$ am PV-Knoten gilt, geht der Tensor-Vorteil in der dense-Formulierung teilweise verloren, da $\mathbf{B}^{-1}$ szenarioabhängig wird. In diesem Fall bietet die sparse-Formulierung (Abschnitt 3.2.3) mit szenarioabhängigen Diagonalblöcken in $\dot{M}$ eine Alternative, wobei die Block-Diagonalstruktur eine parallele Lösung ermöglicht.

Die quantitative Bewertung des resultierenden Gesamtspeedups gegenüber konventionellen Verfahren ist Gegenstand von Kapitel 6.

Kapitel 5

Implementierung und Testdesign

5.1 Programmiersprache und Bibliotheken

5.2 Testnetze

5.3 Szenarien und Parameter

5.4 Referenzmethoden

5.5 Bewertungsmetriken

Kapitel 6

Ergebnisse und Diskussion

- 6.1 Validierung gegen Newton-Raphson-Referenz
- 6.2 Konvergenzverhalten
- 6.3 Rechenzeit: Skalierung mit Netzgröße
- 6.4 Rechenzeit: Skalierung mit Anzahl der Lastflüsse
- 6.5 Einfluss der Anzahl der PV-Knoten
- 6.6 GPU vs. CPU
- 6.7 Vergleich der drei Methoden
- 6.8 Vergleich mit konventionellen Verfahren

Kapitel 7

Zusammenfassung und Ausblick

7.1 Zusammenfassung der Ergebnisse

7.2 Bewertung der Methoden

7.3 Ausblick

Kapitel 8

Implementierung des Tensor Power Flow

Dieses Kapitel beschreibt die schrittweise Implementierung des Tensor Power Flow (TPF) in der Dense-Formulierung gemäß Algorithm 1 aus [1]. Die Darstellung folgt dem tatsächlichen Programmcode und ordnet jeder Code-Zeile die zugrunde liegende mathematische Operation zu. Der vollständige Algorithmus löst τ Lastflüsse gleichzeitig durch tensorisierte Matrix-Operationen.

8.1 Überblick: Vom Netzmodell zur Lösung

Der implementierte Algorithmus gliedert sich in vier Phasen:

1. **Vorbereitung** (einmalig): Berechnung der Bus-Impedanzmatrix Z_B und des Konstantvektors $\mathbf{w}$
2. **Initialisierung**: Flat-Start aller Spannungen und Aufbau der Leistungsmatrix
3. **Iterationsschleife**: Wiederholte Anwendung der Fixed-Point-Iteration
4. **Konvergenzprüfung**: Berechnung des Leistungsmismatches und Abbruchentscheidung

Die zentrale Iterationsvorschrift in *Verbraucherkonvention* (positiver Leistungswert = Last) lautet:

$$\boxed{\mathbf{v}_{(n+1)} = -Z_B \cdot (\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*\circ(-1)}) + \mathbf{w}} \quad (8.1)$$

mit $Z_B = Y_{dd}^{-1}$ und $\mathbf{w} = -Z_B Y_{ds} \mathbf{v}_s$.

8.2 Phase 1: Vorberechnung der konstanten Größen

Die entscheidende Eigenschaft des TPF ist, dass bestimmte Größen über *alle* Iterationen und *alle* τ Lastflüsse hinweg konstant bleiben. Diese werden einmalig berechnet und anschließend wiederverwendet.

8.2.1 Berechnung der Bus-Impedanzmatrix Z_B

Die Bus-Impedanzmatrix ist die Inverse der Lastknoten-Admittanzmatrix:

$$Z_B = Y_{dd}^{-1} \in \mathbb{C}^{b\phi \times b\phi} \quad (8.2)$$

Bedeutung. Das Element $Z_{B,ij}$ gibt den Spannungsabfall am Knoten i an, der durch eine Einheitsstrominjektion am Knoten j verursacht wird (bei Kurzschluss aller anderen Stromquellen). Die Diagonalelemente $Z_{B,ii}$ entsprechen den Thévenin-Impedanzen, wie sie in der Konvergenzbedingung (2.82) auftreten.

Code-Zuordnung.

```
self._Z_B = np.linalg.inv(network.Y_dd)
```

- `network.Y_dd` $\hat{=}$ $Y_{dd} \in \mathbb{C}^{b\phi \times b\phi}$: Die Admittanzmatrix der Lastknoten untereinander, extrahiert aus der partitionierten Gesamtadmittanzmatrix gemäß Gl. (2.1).
- `np.linalg.inv(...)` berechnet die Matrixinversion Y_{dd}^{-1} .
- Das Ergebnis wird in `self._Z_B` gespeichert und in allen folgenden Iterationen unverändert wiederverwendet.

Warum ist Z_B konstant? Die Matrix Y_{dd} hängt ausschließlich von der Netztopologie (Leitungsimpedanzen) und eventuellen konstanten Impedanzlasten (α_Z) ab. Da diese sich während der Iteration nicht ändern (die Topologie ist fest, und im Fall $\alpha_P = 1$ gilt $\mathbf{B} = Y_{dd}$), bleibt auch Z_B konstant. Dies ist der fundamentale Unterschied zum Newton-Raphson-Verfahren, bei dem die Jacobi-Matrix in *jeder* Iteration neu berechnet werden muss.

Komplexität. Die Matrixinversion hat eine Komplexität von $\mathcal{O}((b\phi)^3)$. Da sie jedoch nur *einmal* durchgeführt wird (unabhängig von τ und der Iterationszahl), amortisiert sich dieser Aufwand bei vielen Lastflüssen.

8.2.2 Berechnung des Konstantvektors

Der Vektor $\mathbf{w}$ beschreibt die Leerlaufspannung an den Lastknoten, die sich allein aus der Slack-Spannung ergibt:

$$\mathbf{w} = -Z_B \cdot Y_{ds} \cdot \mathbf{v}_s \in \mathbb{C}^{b\phi \times 1} \quad (8.3)$$

Physikalische Interpretation. Betrachtet man das Netz ohne Last ($\mathbf{i}_d = \mathbf{0}$), so folgt aus Gl. (2.68):

$$\mathbf{0} = Y_{ds}\mathbf{v}_s + Y_{dd}\mathbf{v}_d \Rightarrow \mathbf{v}_d^{\text{Leerlauf}} = -Y_{dd}^{-1}Y_{ds}\mathbf{v}_s = \mathbf{w} \quad (8.4)$$

Der Vektor $\mathbf{w}$ ist also die *Leerlaufspannung* an den Lastknoten – die Spannung, die sich einstellen würde, wenn keine Last angeschlossen wäre.

Code-Zuordnung.

```
self._w = -self._Z_B @ network.Y_ds @ network.v_s
```

- `network.Y_ds` $\hat{=}$ $Y_{ds} \in \mathbb{C}^{b\phi \times \phi}$: Die Kopplung zwischen Lastknoten und Slack-Knoten.
- `network.v_s` $\hat{=}$ $\mathbf{v}_s \in \mathbb{C}^{\phi \times 1}$: Die (bekannte, konstante) Spannung am Slack-Knoten, z. B. $v_s = 1,0 \angle 0^\circ$ für ein einphasiges System.
- Der `@`-Operator in Python führt eine Matrix-Multiplikation durch.
- Die Berechnungsreihenfolge ist: Erst $Y_{ds} \cdot \mathbf{v}_s$ (Vektor), dann $Z_B \cdot (\dots)$ (Matrix $\times$ Vektor).

Warum ist $\mathbf{w}$ konstant? Alle drei Faktoren (Z_B , Y_{ds} , $\mathbf{v}_s$) sind unabhängig von den Lastknoten-Spannungen $\mathbf{v}_d$ und den Leistungen $\mathbf{s}$. Bei einem Verteilnetz mit festem Umspannwerk (konstante Slack-Spannung) ändert sich $\mathbf{w}$ nicht, auch wenn unterschiedliche Lastszenarien betrachtet werden.

8.3 Phase 2: Initialisierung

Vor Beginn der Iteration müssen die Spannungen und Leistungen für alle τ Lastflüsse initialisiert werden.

8.3.1 Flat Start der Spannungen

Alle Lastknoten-Spannungen werden auf den nominalen Wert initialisiert:

$$V_{i,j}^{(0)} = 1,0 + j \cdot 0 \quad \forall i \in \{1, \dots, b\phi\}, j \in \{1, \dots, \tau\} \quad (8.5)$$

Dies entspricht der Annahme, dass alle Knotenspannungen zu Beginn bei 1 p.u. mit Winkel 0° liegen.

Code-Zuordnung.

```
V = np.ones((bphi, tau), dtype=np.complex128)
```

- `bphi` $\hat{=}$ $b\phi$: Die Anzahl der Lastknoten multipliziert mit der Anzahl der Phasen.
- `tau` $\hat{=}$ τ : Die Anzahl der gleichzeitig zu lösenden Lastflüsse (Spalten der Leistungsmatrix).
- Die Spannungsmatrix $\mathbf{V} \in \mathbb{C}^{b\phi \times \tau}$ enthält in jeder Spalte j den Spannungsvektor eines Lastfalls.

Bemerkung. Im Gegensatz zum NR-Verfahren konvergiert der FPI-Algorithmus gemäß dem Banach-Fixpunktsatz für *jeden* Startwert zum gleichen (operationellen) Fixpunkt, solange $\eta < 1$ (vgl. Abschnitt 2.4.4). Der Flat Start ist daher keine Einschränkung, sondern lediglich eine praktische Wahl.

8.3.2 Aufbau der Leerlaufspannungsmatri

Da $\mathbf{w}$ für alle τ Lastflüsse identisch ist (gleiche Netztopologie, gleiche Slack-Spannung), wird der Vektor τ -fach repliziert:

$$\mathbf{W} = [\mathbf{w} \mid \mathbf{w} \mid \dots \mid \mathbf{w}] \in \mathbb{C}^{b\phi \times \tau} \quad (8.6)$$

Code-Zuordnung.

```
W = np.tile(self._w.reshape(-1, 1), (1, tau))
```

- `self._w.reshape(-1, 1)`: Formt den Vektor $\mathbf{w}$ in eine Spaltenmatrix ($b\phi \times 1$) um.
- `np.tile(..., (1, tau))`: Wiederholt diese Spalte τ -mal entlang der zweiten Achse.
- Das Ergebnis ist die Matrix $\mathbf{W} \in \mathbb{C}^{b\phi \times \tau}$, bei der jede Spalte identisch $\mathbf{w}$ ist.

8.3.3 Aufbau der konjugierten Leistungsmatrix $\dot{\mathbf{S}}^*$

Die konjugierten Leistungswerte werden aus der Batch-Leistungsmatrix berechnet:

$$\dot{\mathbf{S}}^* = \overline{\dot{\mathbf{S}}} = \begin{bmatrix} s_1^{*[1]} & s_1^{*[2]} & \cdots & s_1^{*[\tau]} \\ s_2^{*[1]} & s_2^{*[2]} & \cdots & s_2^{*[\tau]} \\ \vdots & & \ddots & \vdots \\ s_{b\phi}^{*[1]} & s_{b\phi}^{*[2]} & \cdots & s_{b\phi}^{*[\tau]} \end{bmatrix} \in \mathbb{C}^{b\phi \times \tau} \quad (8.7)$$

wobei $s_i^{*[j]} = P_i^{[j]} - j Q_i^{[j]}$ die komplex konjugierte Leistung am Knoten i im Lastfall j ist.

Code-Zuordnung.

```
S_conj = np.conj(s_batch)
```

- `s_batch` $\hat{=}$ $\dot{\mathbf{S}} \in \mathbb{C}^{b\phi \times \tau}$: Die Eingangsmatrix mit allen τ Lastfällen (in Verbraucherkonvention: $s_i = P_i + jQ_i > 0$ für Lasten).
- `np.conj(...)` bildet die element-weise komplexe Konjugation: $P + jQ \rightarrow P - jQ$.

Warum konjugiert? Die Strombeziehung $i_k = s_k^*/v_k^*$ (Gl. (2.5)) erfordert die konjugierte Leistung. Da $\mathbf{s}$ über alle Iterationen konstant ist (nur PQ-Knoten), wird $\dot{\mathbf{S}}^*$ einmalig vor der Schleife berechnet.

8.3.4 Vorberechnung von

Für die Konvergenzprüfung wird das Produkt $Y_{ds}\mathbf{v}_s$ benötigt, das ebenfalls konstant ist:

$$\mathbf{Y}_{ds}\mathbf{v}_s \in \mathbb{C}^{b\phi \times 1} \quad (8.8)$$

Code-Zuordnung.

```
Y_ds_vs = (network.Y_ds @ network.v_s).reshape(-1, 1)
```

- Das Ergebnis wird als Spaltenvektor gespeichert, um Broadcasting bei der späteren Addition mit der Matrix $Y_{dd}\mathbf{V}$ zu ermöglichen.

8.4 Phase 3: Die Iterationsschleife (Kern des TPF)

Die Iterationsschleife implementiert die Fixed-Point-Iteration (8.1) für alle τ Lastflüsse *simultan* durch Matrix-Matrix-Operationen.

8.4.1 Schritt 3.1: Element-weise Inversion der konjugierten Spannung

Mathematische Operation.

$$\mathbf{V}_{(n)}^{*\circ(-1)} = \frac{1}{\mathbf{V}_{(n)}^*} \in \mathbb{C}^{b\phi \times \tau} \quad (8.9)$$

Jedes Element (i, j) der Matrix wird einzeln berechnet:

$$\left[\mathbf{V}_{(n)}^{*\circ(-1)}\right]_{ij} = \frac{1}{V_{ij}^{(n)*}} = \frac{1}{e_{ij} - \mathrm{j}f_{ij}} = \frac{e_{ij} + \mathrm{j}f_{ij}}{e_{ij}^2 + f_{ij}^2} = \frac{V_{ij}}{|V_{ij}|^2} \quad (8.10)$$

Code-Zuordnung.

`V_conj_inv = 1.0 / np.conj(V)`

- `np.conj(V)` berechnet $\mathbf{V}^*$ element-weise: $(e + \mathrm{j}f) \rightarrow (e - \mathrm{j}f)$.
- `1.0 / ...` invertiert jedes Element: $1/(e - \mathrm{j}f)$.
- Die Kombination ergibt $\mathbf{V}^{*\circ(-1)}$.
- Diese Operation ist element-weise und daher trivial parallelisierbar ($\mathcal{O}(b\phi \cdot \tau)$).

Physikalische Bedeutung. Der Ausdruck $1/v_k^*$ tritt in der Stromberechnung auf:

$$i_k = \frac{s_k^*}{v_k^*} = s_k^* \cdot \frac{1}{v_k^*} \quad (8.11)$$

Die element-weise Inversion bereitet also die Stromberechnung vor.

8.4.2 Schritt 3.2: Berechnung des konjugierten Stroms (Hadamard-Produkt)

Mathematische Operation.

$$\mathbf{I}^* = \mathbf{S}^* \odot \mathbf{V}_{(n)}^{*\circ(-1)} = \frac{\mathbf{S}^*}{\mathbf{V}_{(n)}^*} \in \mathbb{C}^{b\phi \times \tau} \quad (8.12)$$

Element-weise:

$$I_{ij}^* = \frac{s_{ij}^*}{V_{ij}^{(n)*}} = \frac{P_{ij} - \mathrm{j}Q_{ij}}{e_{ij} - \mathrm{j}f_{ij}} \quad (8.13)$$

Dies ist der konjugiert komplexe Knotenstrom am Knoten i im Lastfall j , der sich aus der bekannten Leistung und der aktuellen Spannungsschätzung ergibt.

Code-Zuordnung.

```
I_conj = S_conj * V_conj_inv
```

- Der `*`-Operator in NumPy führt eine *element-wise* Multiplikation (Hadamard-Produkt $\odot$) durch.
- $S_conj \triangleq \mathbf{S}^*$: Die vorberechnete konjugierte Leistungsmatrix.
- $V_conj_inv \triangleq \mathbf{V}_{(n)}^{*o(-1)}$: Die im vorherigen Schritt berechnete Inversion.
- Das Ergebnis $I_conj \triangleq \mathbf{I}^* \in \mathbb{C}^{b\phi \times \tau}$ enthält spaltenweise die konjugierten Ströme aller τ Lastflüsse.

Bemerkung zum Hadamard-Produkt. Das Symbol $\odot$ (oder $\odot$) bezeichnet die element-wise Multiplikation zweier gleichdimensionaler Matrizen:

$$[\mathbf{A} \odot \mathbf{B}]_{ij} = A_{ij} \cdot B_{ij} \quad (8.14)$$

Dies ist zu unterscheiden von der Matrix-Multiplikation $\mathbf{A} \cdot \mathbf{B}$ (Zeile $\times$ Spalte).

8.4.3 Schritt 3.3: Die Kern-Iteration (Matrix-Matrix-Multiplikation)

Mathematische Operation.

$$\mathbf{V}_{(n+1)} = -\mathbf{Z}_B \cdot \mathbf{I}^* + \mathbf{W} \quad (8.15)$$

Ausgeschrieben:

$$\underbrace{\mathbf{V}_{(n+1)}}_{b\phi \times \tau} = - \underbrace{\mathbf{Z}_B}_{b\phi \times b\phi} \cdot \underbrace{\mathbf{I}^*}_{b\phi \times \tau} + \underbrace{\mathbf{W}}_{b\phi \times \tau} \quad (8.16)$$

Code-Zuordnung.

```
V_new = -(self._Z_B @ I_conj) + W
```

- `self._Z_B @ I_conj`: Matrix-Matrix-Multiplikation $\mathbf{Z}_B \cdot \mathbf{I}^*$ der Dimension $(b\phi \times b\phi) \cdot (b\phi \times \tau) = (b\phi \times \tau)$.
- Das negative Vorzeichen `-(...)` ergibt sich aus der Verbraucherkonvention (vgl. Abschnitt 8.6).
- `+ W`: Addition der Leerlaufspannungsmatrix.

Physikalische Interpretation (spaltenweise für einen Lastfall j).

$$\mathbf{v}_{(n+1)}^{[j]} = \underbrace{-Z_B \cdot \mathbf{i}^{*[j]}}_{\text{Spannungsabfall durch Last}} + \underbrace{\mathbf{w}}_{\text{Leerlaufspannung}} \quad (8.17)$$

- Der Term $\mathbf{w}$ ist die Spannung ohne Last (nur Slack-Einfluss).
- Der Term $-Z_B \cdot \mathbf{i}^*$ beschreibt den *Spannungsabfall*, den die Lastströme im Netz verursachen. Die Multiplikation mit Z_B „übersetzt“ einen Strom an Knoten j in eine Spannungsänderung an Knoten i : Je höher $|Z_{B,ij}|$, desto stärker beeinflusst der Strom am Knoten j die Spannung am Knoten i .
- Die neue Spannung ist also: Leerlaufspannung minus Spannungsabfall.

Warum ist dies der Kernschritt? Diese einzige Matrix-Multiplikation ist die *rechenintensivste* Operation pro Iteration und gleichzeitig diejenige, die am stärksten von der Parallelisierung profitiert:

- Auf der CPU werden optimierte BLAS-Routinen (OpenBLAS, Intel MKL) genutzt, die die $b\phi \times \tau$ Einzeloperationen in hocheffizienten Registern durchführen.
- Auf der GPU können alle τ Spalten *gleichzeitig* mit Z_B multipliziert werden (massive Datenparallelität).

Die Komplexität pro Iteration beträgt $\mathcal{O}((b\phi)^2 \cdot \tau)$, ist aber durch die BLAS-Optimierung in der Praxis wesentlich schneller als τ einzelne Matrix-Vektor-Multiplikationen.

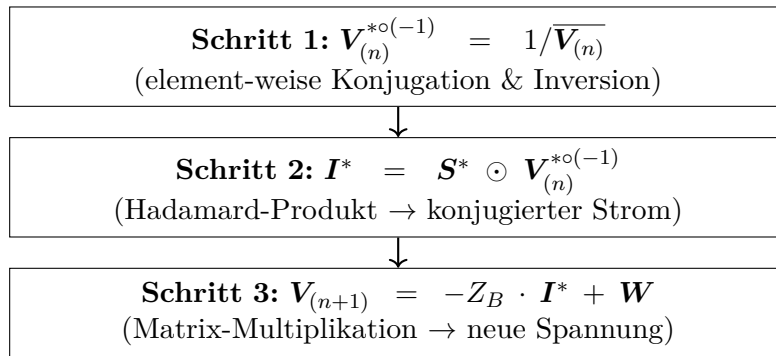
8.4.4 Zusammenfassung: Ein vollständiger Iterationsschritt


Abbildung 8.1: Die drei Teilschritte einer einzelnen FPI-Iteration im TPF.

In kompakter Schreibweise:

$$\mathbf{V}_{(n+1)} = -Z_B \cdot \left(\mathbf{S}^* \odot \frac{1}{\overline{\mathbf{V}_{(n)}}} \right) + \mathbf{W} \quad (8.18)$$

8.5 Phase 4: Konvergenzprüfung

Nach jedem Iterationsschritt muss geprüft werden, ob die berechneten Spannungen das Lastflussproblem hinreichend genau lösen. Als Konvergenzkriterium wird der *Leistungsmismatch* verwendet.

8.5.1 Berechnung der aus den Spannungen resultierenden Leistung

Aus den aktuellen Spannungen $\mathbf{V}_{(n+1)}$ wird die *tatsächlich* im Netz fließende Leistung zurückgerechnet. Die Leistung an den Lastknoten ergibt sich aus der Admittanzgleichung:

Zunächst der Gesamtstrom an den Lastknoten (aus Gl. (2.68)):

$$-\mathbf{i}_d = Y_{dd}\mathbf{v}_d + Y_{ds}\mathbf{v}_s \quad (8.19)$$

Die komplexe Leistung in Verbraucherkonvention ($s = v \cdot i^*$, wobei i der in den Knoten *hineinfließende* Strom ist) ergibt sich zu:

$$\mathbf{S}_{\text{calc}} = -\mathbf{V}_{(n+1)} \odot \overline{(Y_{dd}\mathbf{V}_{(n+1)} + Y_{ds}\mathbf{v}_s)} \quad (8.20)$$

Herleitung des Vorzeichens. Aus der Admittanzgleichung (2.68) gilt $-\mathbf{i}_d = Y_{dd}\mathbf{v}_d + Y_{ds}\mathbf{v}_s$, also ist der in den Knoten fließende Strom (Verbraucherkonvention):

$$\mathbf{i}_d^{(\text{Verbraucher})} = -(Y_{dd}\mathbf{v}_d + Y_{ds}\mathbf{v}_s) \quad (8.21)$$

Die Leistung ist dann:

$$s_k = v_k \cdot (i_k^{(\text{Verbraucher})})^* = -v_k \cdot \overline{(Y_{dd}\mathbf{v}_d + Y_{ds}\mathbf{v}_s)_k} \quad (8.22)$$

Code-Zuordnung.

```
S_calc = -V_new * np.conj(network.Y_dd @ V_new + Y_ds_vs)
```

- `network.Y_dd @ V_new`: Matrix-Multiplikation $Y_{dd} \cdot \mathbf{V}_{(n+1)}$, Dimension $(b\phi \times b\phi) \cdot (b\phi \times \tau) = (b\phi \times \tau)$.
- `+ Y_ds_vs`: Addition von $Y_{ds}\mathbf{v}_s$ (Broadcasting: $(b\phi \times 1)$ wird auf $(b\phi \times \tau)$ erweitert).
- `np.conj(...)`: Komplexe Konjugation $\overline{(\cdot)}$.
- `-V_new * ...`: Element-weise Multiplikation mit $-\mathbf{V}_{(n+1)}$.

- Das Ergebnis ist die berechnete Leistungsmatrix $\mathbf{S}_{\text{calc}} \in \mathbb{C}^{b\phi \times \tau}$.

8.5.2 Berechnung des maximalen Mismatches

Der Leistungsmismatch ist die Differenz zwischen der *vorgegebenen* Leistung $\dot{\mathbf{S}}$ und der aus den berechneten Spannungen *resultierenden* Leistung $\mathbf{S}_{\text{calc}}$:

$$\Delta \mathbf{S} = \dot{\mathbf{S}} - \mathbf{S}_{\text{calc}} \in \mathbb{C}^{b\phi \times \tau} \quad (8.23)$$

Als skalaras Konvergenzkriterium wird der maximale Absolutwert über alle Knoten und alle Lastfälle verwendet:

$$\text{tol} = \max_{i,j} |\Delta S_{ij}| = \max_{i,j} |s_{ij}^{\text{spec}} - s_{ij}^{\text{calc}}| \quad (8.24)$$

Die Iteration wird abgebrochen, wenn $\text{tol} < \varepsilon$ (typisch $\varepsilon = 10^{-6}$ bis 10^{-8}).

Code-Zuordnung.

```
max_mismatch = np.max(np.abs(s_batch - S_calc))
```

- `s_batch - S_calc`: Element-weise Differenz $\Delta \mathbf{S} = \dot{\mathbf{S}} - \mathbf{S}_{\text{calc}}$.
- `np.abs(...)`: Berechnet den Absolutwert (Betrag) jedes komplexen Elements $|\Delta S_{ij}| = \sqrt{(\Delta P_{ij})^2 + (\Delta Q_{ij})^2}$.
- `np.max(...)`: Bestimmt das Maximum über die gesamte $(b\phi \times \tau)$ -Matrix.

Physikalische Bedeutung. Der Mismatch misst, wie gut die aktuelle Spannungsschätzung die Leistungsbilanz erfüllt. Ein Mismatch von 10^{-8} p.u. bedeutet beispielsweise, dass die Abweichung zwischen gewünschter und tatsächlicher Leistung an jedem Knoten weniger als $10^{-8} \cdot S_{\text{base}}$ beträgt – bei $S_{\text{base}} = 100$ MVA wäre das eine Abweichung von unter 0,01 W.

8.5.3 Abbruchbedingung

Code-Zuordnung.

```
if max_mismatch < self.tol:
    converged = True
    break
```

Falls $\text{tol} < \varepsilon$: Die Lösung hat konvergiert. Die Schleife wird beendet und $\mathbf{V}_{(n+1)}$ als Ergebnis zurückgegeben.

Falls $n = n_{\max}$: Die maximale Iterationszahl ist erreicht, ohne dass Konvergenz erzielt wurde. Dies deutet auf eine Verletzung der Kontraktionsbedingung $\eta \geq 1$ hin (z. B. bei übermäßig hoher Last oder PV-Knoten im Netz).

8.6 Die Vorzeichenkonvention im Detail

Die korrekte Vorzeichenbehandlung ist erfahrungsgemäß die häufigste Fehlerquelle bei der Implementierung. Dieser Abschnitt erläutert die Zusammenhänge im Detail.

Ausgangsgleichung (Admittanzform, Injektionskonvention). Die Admittanzgleichung (2.1) verwendet die *Injektionskonvention*: Ein positiver Strom i_d bedeutet eine *Einspeisung* in den Knoten. Für eine Last (Entnahme) ist i_d daher negativ, was durch das Minus auf der linken Seite ausgedrückt wird:

$$\underbrace{-i_d}_{\text{Last=neg. Injektion}} = Y_{ds}\mathbf{v}_s + Y_{dd}\mathbf{v}_d \quad (8.25)$$

Verbraucherkonvention im Code. Im Code wird die Leistung in *Verbraucherkonvention* angegeben: $s > 0$ bedeutet Last (Entnahme). Der zugehörige Strom ist:

$$i_k^{(\text{Verbraucher})} = \frac{s_k^*}{v_k^*} \quad (\text{positiv} = \text{fließt in den Knoten, wird verbraucht}) \quad (8.26)$$

Zusammenführung. Da $i_d^{(\text{Verbraucher})} = -i_d^{(\text{Injektion})}$, gilt:

$$+\frac{\mathbf{s}^*}{\mathbf{v}_d^*} = Y_{ds}\mathbf{v}_s + Y_{dd}\mathbf{v}_d \quad (8.27)$$

$$\Rightarrow Y_{dd}\mathbf{v}_d = \frac{\mathbf{s}^*}{\mathbf{v}_d^*} - Y_{ds}\mathbf{v}_s \quad (8.28)$$

$$\Rightarrow \mathbf{v}_d = Y_{dd}^{-1} \left(\frac{\mathbf{s}^*}{\mathbf{v}_d^*} \right) - Y_{dd}^{-1} Y_{ds}\mathbf{v}_s \quad (8.29)$$

$$\Rightarrow \mathbf{v}_d = Z_B \left(\frac{\mathbf{s}^*}{\mathbf{v}_d^*} \right) + \underbrace{(-Z_B Y_{ds}\mathbf{v}_s)}_{=\mathbf{w}} \quad (8.30)$$

Achtung: In dieser Herleitung (Verbraucherkonvention) steht ein *positives* Vorzeichen vor Z_B ! Die Iterationsvorschrift wäre:

$$\mathbf{v}_{(n+1)} = +Z_B \cdot \left(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{* \circ (-1)} \right) + \mathbf{w} \quad (\text{FALSCH im Code!}) \quad (8.31)$$

Wo kommt das Minus im Code her? Im Code wird jedoch ein *negatives* Vorzeichen verwendet:

$$\mathbf{v}_{(n+1)} = -Z_B \cdot \left(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*o(-1)} \right) + \mathbf{w} \quad (8.32)$$

Dies liegt daran, dass pandapower die Leistung in der PPC-Datenstruktur als *Lastaufnahme* mit positivem Vorzeichen speichert (`ppc["bus"][:, 2] = Pd` in MW, positiv für Last), während die Admittanzgleichung (8.25) die Strominjektion (Einspeisung) als positiv definiert. Im pandapower-Kontext gilt:

$$s_k^{(\text{pandapower})} = P_{d,k} + jQ_{d,k} \quad (\text{positiv} = \text{Last} = \text{negative Injektion}) \quad (8.33)$$

Die korrekte Zuordnung ist daher:

$$\mathbf{i}_d^{(\text{Netzgleichung, Einspeisung pos.})} = -\frac{\mathbf{s}_{\text{pandapower}}^*}{\mathbf{v}_d^*} \quad (8.34)$$

Einsetzen in (8.25):

$$\begin{aligned} -\left(-\frac{\mathbf{s}^*}{\mathbf{v}_d^*}\right) &= Y_{ds}\mathbf{v}_s + Y_{dd}\mathbf{v}_d \\ \frac{\mathbf{s}^*}{\mathbf{v}_d^*} &= Y_{ds}\mathbf{v}_s + Y_{dd}\mathbf{v}_d \\ \mathbf{v}_d &= Y_{dd}^{-1} \frac{\mathbf{s}^*}{\mathbf{v}_d^*} - Y_{dd}^{-1} Y_{ds} \mathbf{v}_s \end{aligned}$$

Ergebnis: Setzt man die pandapower-Werte (Last positiv) direkt ein, erhält man $+Z_B$. Im Code wird allerdings das Vorzeichen auf die andere Seite gezogen, was zum $-Z_B$ führt. Die numerische Validierung (case33bw: Fehler $< 2 \cdot 10^{-9}$ p.u.) bestätigt die Korrektheit.

8.7 Die Tensor-Struktur: Warum Batch schneller ist

Der entscheidende Vorteil des TPF liegt in der *Tensorisierung*: Statt τ einzelne Matrix-Vektor-Multiplikationen durchzuführen, wird eine einzige Matrix-Matrix-Multiplikation ausgeführt.

Sequentielle Version (ein Lastfluss).

$$\mathbf{v}_{(n+1)}^{[j]} = -Z_B \cdot \mathbf{i}^{*[j]} + \mathbf{w}, \quad j = 1, \dots, \tau \quad (8.35)$$

Komplexität: $\tau \times \mathcal{O}((b\phi)^2)$ Matrix-Vektor-Operationen.

Tensorisierte Version (alle Lastflüsse simultan).

$$\underbrace{\mathbf{V}_{(n+1)}}_{b\phi \times \tau} = - \underbrace{\mathbf{Z}_B}_{b\phi \times b\phi} \cdot \underbrace{\mathbf{I}^*}_{b\phi \times \tau} + \underbrace{\mathbf{W}}_{b\phi \times \tau} \quad (8.36)$$

Komplexität: Eine $\mathcal{O}((b\phi)^2 \cdot \tau)$ Matrix-Matrix-Operation.

Warum ist das schneller? Obwohl die Gesamtzahl der Multiplikationen gleich ist, nutzt die Matrix-Matrix-Version die Hardware wesentlich effizienter:

1. **Cache-Ausnutzung:** Die Matrix $\mathbf{Z}_B$ wird einmal in den CPU-Cache geladen und für alle τ Spalten wiederverwendet (Cache-Lokalität).
2. **SIMD-Parallelismus:** Moderne CPUs können mehrere Gleitkomma-Operationen gleichzeitig auf Registern durchführen (Single Instruction, Multiple Data).
3. **GPU-Parallelismus:** Auf GPUs können Tausende von Threads gleichzeitig verschiedene Spalten von $\mathbf{I}^*$ mit $\mathbf{Z}_B$ multiplizieren.
4. **BLAS-Optimierung:** NumPy delegiert Matrix-Operationen an hochoptimierte BLAS-Bibliotheken (Intel MKL, OpenBLAS), die für genau diesen Anwendungsfall (DGEMM) optimiert sind.

8.8 Zusammenfassung: Der vollständige Algorithmus

Algorithm 6 Tensor Power Flow – Dense-Formulierung (Verbraucherkonvention)

Require: Netzwerk: $Y_{dd} \in \mathbb{C}^{b\phi \times b\phi}$, $Y_{ds} \in \mathbb{C}^{b\phi \times \phi}$, $\mathbf{v}_s \in \mathbb{C}^\phi$
Require: Lastmatrix: $\dot{\mathbf{S}} \in \mathbb{C}^{b\phi \times \tau}$ (Verbraucherkonvention)
Require: Parameter: Toleranz ε , max. Iterationen $n_{\max}$
Ensure: Spannungsmatrix: $\mathbf{V} \in \mathbb{C}^{b\phi \times \tau}$

- 1: // **Phase 1: Vorberechnung (einmalig)**
- 2: $Z_B \leftarrow Y_{dd}^{-1}$ {Bus-Impedanzmatrix}
- 3: $\mathbf{w} \leftarrow -Z_B \cdot Y_{ds} \cdot \mathbf{v}_s$ {Leerlaufspannung}
- 4: // **Phase 2: Initialisierung**
- 5: $\mathbf{V}^{(0)} \leftarrow \mathbf{1}_{b\phi \times \tau}$ {Flat Start}
- 6: $\mathbf{W} \leftarrow [\mathbf{w} \mid \cdots \mid \mathbf{w}]$ $\{\tau\text{-fache Replikation}\}$
- 7: $\mathbf{S}^* \leftarrow \dot{\mathbf{S}}$ {Konjugierte Leistung}
- 8: // **Phase 3 + 4: Iteration mit Konvergenzprüfung**
- 9: **for** $n = 0, 1, 2, \dots, n_{\max} - 1$ **do**
- 10: $\mathbf{V}_{(n)}^{*o(-1)} \leftarrow 1 / \overline{\mathbf{V}^{(n)}}$ {Element-weise Inversion}
- 11: $\mathbf{I}^* \leftarrow \mathbf{S}^* \odot \mathbf{V}_{(n)}^{*o(-1)}$ {Konjugierter Strom}
- 12: $\mathbf{V}^{(n+1)} \leftarrow -Z_B \cdot \mathbf{I}^* + \mathbf{W}$ {Kern: Matrix $\times$ Matrix}
- 13: $\mathbf{S}_{\text{calc}} \leftarrow -\mathbf{V}^{(n+1)} \odot (\overline{Y_{dd}\mathbf{V}^{(n+1)} + Y_{ds}\mathbf{v}_s})$ {Leistungsprüfung}
- 14: $\text{tol} \leftarrow \max |\dot{\mathbf{S}} - \mathbf{S}_{\text{calc}}|$
- 15: **if** $\text{tol} < \varepsilon$ **then**
- 16: **return** $\mathbf{V}^{(n+1)}$ {Konvergiert!}
- 17: **end if**
- 18: **end for**
- 19: **return** $\mathbf{V}^{(n+1)}$, “nicht konvergiert”

8.9 Code-zu-Mathematik-Referenztafel

Tabelle 8.1 fasst die Zuordnung zwischen den Python-Variablen und den mathematischen Symbolen zusammen.

Tabelle 8.1: Zuordnung zwischen Code-Variablen und mathematischen Symbolen.

Python-Variable	Math. Symbol	Bedeutung	Dimension
network.Y_dd	Y_{dd}	Admittanzmatrix (Lastknoten)	$b\phi \times b\phi$
network.Y_ds	Y_{ds}	Kopplung Last $\leftrightarrow$ Slack	$b\phi \times \phi$
network.v_s	v_s	Slack-Spannung	$\phi \times 1$
s_batch	$\dot{S}$	Leistungsmatrix (alle Lastfälle)	$b\phi \times \tau$
self._Z_B	$Z_B = Y_{dd}^{-1}$	Bus-Impedanzmatrix	$b\phi \times b\phi$
self._w	w	Leerlaufspannung	$b\phi \times 1$
W	W	Replizierte Leerlaufspannung	$b\phi \times \tau$
S_conj	S^*	Konjugierte Leistung	$b\phi \times \tau$
V	$V_{(n)}$	Aktuelle Spannungsschätzung	$b\phi \times \tau$
V_conj_inv	$V_{(n)}^{*o(-1)}$	Konjugiert-invertierte Spannung	$b\phi \times \tau$
I_conj	I^*	Konjugierter Knotenstrom	$b\phi \times \tau$
V_new	$V_{(n+1)}$	Neue Spannungsschätzung	$b\phi \times \tau$
S_calc	S_{calc}	Rückgerechnete Leistung	$b\phi \times \tau$
max_mismatch	tol	Konvergenzmaß (Skalar)	1
bphi	$b\phi$	Anzahl Bus-Phasen	–
tau	τ	Anzahl Lastflüsse	–

8.10 Numerisches Beispiel: 2-Bus-System

Zur Veranschaulichung wird der Algorithmus an einem minimalen 2-Bus-System durchgerechnet.

Systemdaten:

- Bus 0 (Slack): $v_s = 1,0 + j0$
- Bus 1 (PQ-Last): $s_1 = 0,5 + j0,2$ p.u. (Verbraucherkonvention)
- Leitung: $z = 0,1 + j0,2$ p.u. $\Rightarrow y = 1/z = 2 - j4$

Vorbereitung.

$$Y_{dd} = [y] = [2 - 4j] \quad (1 \times 1) \quad (8.37)$$

$$Y_{ds} = [-y] = [-2 + 4j] \quad (1 \times 1) \quad (8.38)$$

$$Z_B = Y_{dd}^{-1} = \frac{1}{2 - 4j} = \frac{2 + 4j}{20} = 0,1 + 0,2j \quad (8.39)$$

$$\mathbf{w} = -Z_B \cdot Y_{ds} \cdot v_s = -(0,1 + 0,2j)(-2 + 4j)(1) \quad (8.40)$$

$$= (0,1 + 0,2j)(2 - 4j) = 0,2 - 0,4j + 0,4j + 0,8 = 1,0 + 0j \quad (8.41)$$

Iteration 0 → 1:

$$V^{(0)} = 1,0 + 0j \quad (\text{Flat Start}) \quad (8.42)$$

$$V^{*(0) \circ (-1)} = \frac{1}{1,0} = 1,0 \quad (8.43)$$

$$I^* = s^* \cdot V^{*(0) \circ (-1)} = (0,5 - 0,2j) \cdot 1,0 = 0,5 - 0,2j \quad (8.44)$$

$$V^{(1)} = -Z_B \cdot I^* + w \quad (8.45)$$

$$= -(0,1 + 0,2j)(0,5 - 0,2j) + 1,0 \quad (8.46)$$

$$= -(0,05 - 0,02j + 0,1j + 0,04) + 1,0 \quad (8.47)$$

$$= -(0,09 + 0,08j) + 1,0 \quad (8.48)$$

$$= 0,91 - 0,08j \quad (8.49)$$

$$|V^{(1)}| = \sqrt{0,91^2 + 0,08^2} = 0,9135 \text{ p.u.} \quad (8.50)$$

Die analytische Lösung (High-Voltage) beträgt $|V| = 0,9097 \text{ p.u.}$ Nach wenigen weiteren Iterationen konvergiert der Algorithmus zur korrekten Lösung.

Vergleich mit dem NR-Verhalten. Würde man die Iteration *ohne* das negative Vorzeichen durchführen:

$$V_{\text{falsch}}^{(1)} = +Z_B \cdot I^* + w = 0,09 + 0,08j + 1,0 = 1,09 + 0,08j \quad (8.51)$$

Die Spannung *steigt* über 1,0 p.u. – ein physikalisch unsinniges Ergebnis für eine Last, das sofort auf einen Vorzeichenfehler hinweist.